

NYC Yellow Taxi Trip Data Analysis

Project Objective

Analyze NYC Yellow Taxi trip data for January 2025 to understand trip patterns, distance, duration, passenger behavior, time trends, locations, payment types, vendors, and rate codes.

Name: Dhruv Sanjay Ghugal
Degree: B.Tech Computer Engineering
Dataset: NYC Yellow Taxi Trip Data — January 2025
Tools: Python, NumPy, Pandas, Matplotlib, Seaborn

Analysis Workflow

1. Raw Data Understanding
2. Data Cleaning
3. Feature Engineering
4. Exploratory Data Analysis
5. Statistical Analysis
6. Business Questions
7. Advanced Analysis
8. Key Findings & Conclusion

```
#importing library as keywords
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Phase A — Data Understanding & Quality Assessment

```
url = "https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2025-01.parquet"

main = pd.read_parquet(url)
main
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fw
0	1	2025-01-01 00:18:38	2025-01-01 00:26:59	1.0	1.60	1.0	
1	1	2025-01-01 00:32:40	2025-01-01 00:35:13	1.0	0.50	1.0	
2	1	2025-01-01 00:44:04	2025-01-01 00:46:01	1.0	0.60	1.0	
3	2	2025-01-01 00:14:27	2025-01-01 00:20:01	3.0	0.52	1.0	
4	2	2025-01-01 00:21:34	2025-01-01 00:25:06	3.0	0.66	1.0	
...	...	...	...	...	...	...	
3475221	2	2025-01-31 23:01:48	2025-01-31 23:16:29	NaN	3.35	NaN	
3475222	2	2025-01-31 23:50:29	2025-02-01 00:17:27	NaN	8.73	NaN	
3475223	2	2025-01-31 23:26:59	2025-01-31 23:43:01	NaN	2.64	NaN	
3475224	2	2025-01-31 23:14:34	2025-01-31 23:34:52	NaN	3.16	NaN	
3475225	2	2025-01-31 23:56:42	2025-02-01 00:07:27	NaN	2.29	NaN	

3475226 rows × 20 columns

```
#Task 15
#imported the dataset of NYC Yellow Taxi Trip
#   main= pd.read_parquet('/content/yellow_tripdata_2025-01.parquet')
```

```
#task 16
main.head()
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag
0	1	2025-01-01 00:18:38	2025-01-01 00:26:59	1.0	1.60	1.0	N
1	1	2025-01-01 00:32:40	2025-01-01 00:35:13	1.0	0.50	1.0	N
2	1	2025-01-01 00:44:04	2025-01-01 00:46:01	1.0	0.60	1.0	N
3	2	2025-01-01 00:14:27	2025-01-01 00:20:01	3.0	0.52	1.0	N
4	2	2025-01-01 00:21:34	2025-01-01 00:25:06	3.0	0.66	1.0	N

```
main.tail()
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fw
3475221	2	2025-01-31 23:01:48	2025-01-31 23:16:29	NaN	3.35	NaN	
3475222	2	2025-01-31 23:50:29	2025-02-01 00:17:27	NaN	8.73	NaN	
3475223	2	2025-01-31 23:26:59	2025-01-31 23:43:01	NaN	2.64	NaN	
3475224	2	2025-01-31 23:14:34	2025-01-31 23:34:52	NaN	3.16	NaN	
3475225	2	2025-01-31 23:56:42	2025-02-01 00:07:27	NaN	2.29	NaN	

```
#Task 17
print(main.shape)
```

```
(3475226, 20)
```

```
#Task 18
print(main.columns)          #Given us the name of all the columsn present
```

```
Index(['VendorID', 'tpep_pickup_datetime', 'tpep_dropoff_datetime',
       'passenger_count', 'trip_distance', 'RatecodeID', 'store_and_fwd_flag',
       'PULocationID', 'DOLocationID', 'payment_type', 'fare_amount', 'extra',
       'mta_tax', 'tip_amount', 'tolls_amount', 'improvement_surcharge',
       'total_amount', 'congestion_surcharge', 'Airport_fee',
       'cbd_congestion_fee'],
      dtype='object')
```

```
#Task 19
main.info()         #Gives dtypes and  not null values
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3475226 entries, 0 to 3475225
```

```
Data columns (total 20 columns):
 #   Column                 Dtype
---  ------                 -----
 0   VendorID               int32
 1   tpep_pickup_datetime   datetime64[us]
 2   tpep_dropoff_datetime  datetime64[us]
 3   passenger_count        float64
 4   trip_distance          float64
 5   RatecodeID             float64
 6   store_and_fwd_flag     object
 7   PULocationID           int32
 8   DOLocationID           int32
 9   payment_type           int64
 10  fare_amount            float64
 11  extra                  float64
 12  mta_tax                float64
 13  tip_amount             float64
 14  tolls_amount           float64
 15  improvement_surcharge  float64
 16  total_amount           float64
 17  congestion_surcharge   float64
 18  Airport_fee            float64
 19  cbd_congestion_fee     float64
dtypes: datetime64[us](2), float64(13), int32(3), int64(1), object(1)
memory usage: 490.5+ MB
```

```
#Task 20
print(main.nunique())
```

```
VendorID                      4
tpep_pickup_datetime    1672077
tpep_dropoff_datetime   1671993
passenger_count              10
trip_distance              4545
RatecodeID                    7
store_and_fwd_flag            2
PULocationID                261
DOLocationID                260
payment_type                  6
fare_amount                11538
extra                         77
mta_tax                       9
tip_amount                 4197
tolls_amount               1234
improvement_surcharge         4
total_amount               21995
congestion_surcharge          3
Airport_fee                   7
cbd_congestion_fee            3
dtype: int64
```

```
#Task 21
print(main.isnull().sum())
```

```
VendorID                      0
tpep_pickup_datetime          0
tpep_dropoff_datetime         0
passenger_count          540149
trip_distance                 0
RatecodeID               540149
store_and_fwd_flag       540149
PULocationID                  0
DOLocationID                  0
payment_type                  0
fare_amount                   0
extra                         0
mta_tax                       0
tip_amount                    0
tolls_amount                  0
improvement_surcharge         0
total_amount                  0
congestion_surcharge     540149
Airport_fee              540149
cbd_congestion_fee            0
dtype: int64
```

```
#Task 22
print((main.isnull().sum() / len(main)) * 100)
```

```
VendorID                  0.000000
tpep_pickup_datetime      0.000000
tpep_dropoff_datetime     0.000000
passenger_count          15.542845
trip_distance             0.000000
RatecodeID               15.542845
store_and_fwd_flag       15.542845
PULocationID              0.000000
```

```
DOLocationID              0.000000
payment_type              0.000000
fare_amount               0.000000
extra                     0.000000
mta_tax                   0.000000
tip_amount                0.000000
tolls_amount              0.000000
improvement_surcharge     0.000000
total_amount              0.000000
congestion_surcharge     15.542845
Airport_fee              15.542845
cbd_congestion_fee        0.000000
dtype: float64
```

```
#Task 23
print(main.duplicated().sum(), 'number of duplicated rows')
```

```
0 number of duplicated rows
```

```
#Task 24
print(main[['tpep_pickup_datetime', 'tpep_dropoff_datetime']].dtypes ) #To show dtype of pickup an drop
#those are stored in datetime64[us] format
```

```
tpep_pickup_datetime     datetime64[us]
tpep_dropoff_datetime    datetime64[us]
dtype: object
```

```
#Task 25
print((main['trip_distance'] <= 0).sum(), 'records are Zero/negative distances')
```

```
90893 records are Zero/negative distances
```

```
#Task 26
print((main['passenger_count'] <= 0).sum(), 'records')
```

```
24656 records
```

```
#Task 27
print((main['tpep_dropoff_datetime'] < main['tpep_pickup_datetime']).sum(),'drop-off time occurs before p
```

```
124 drop-off time occurs before pickup time
```

```
#Task 28
# Checking pickup time = drop-off time
zero_duration = (main['tpep_dropoff_datetime'] == main['tpep_pickup_datetime'])
print(zero_duration.sum(), 'records')
```

```
1927 records
```

```
#Task 29
trip_duration = ((main['tpep_dropoff_datetime'] - main['tpep_pickup_datetime']).dt.total_seconds() / 60)

trip_duration.describe()
#The 10 longest trip durations
print(trip_duration.nlargest(10))
```

```
1858816    5626.316667
1858817    5626.316667
414374     5022.133333
2460536    4184.000000
898201     3405.466667
2147536    3062.383333
85681      2876.500000
1536147    2875.166667
556772     2764.533333
3417048    2589.333333
dtype: float64
```

```
#Task 30
# The 10 highest trip distances
print(main.loc[main['trip_distance'].nlargest(10).index,['trip_distance']] )
```

```
         trip_distance
3275762      276423.57
3188501      276099.95
3240337      222167.49
3112173      206137.99
```

```
3374854      202771.63
3081597      189687.43
3069039      181139.99
3312563      168079.57
3195037      167452.94
3138510      164959.95
```

```
#Task 31
# Unique values in VendorID
print(main['VendorID'].value_counts(dropna=False))      #have used dropna=False to get NaN values
print("\n" + "---" * 20 + "\n")

# Unique values in RatecodeID
print(main['RatecodeID'].value_counts(dropna=False))
print("\n" + "---" * 20 + "\n")

# Unique values in store_and_fwd_flag
print(main['store_and_fwd_flag'].value_counts(dropna=False))
print("\n" + "---" * 20 + "\n")

# Unique values in payment_type
print(main['payment_type'].value_counts(dropna=False))
```

```
VendorID
2    2719860
1     753671
7       1206
6        489
Name: count, dtype: int64

------------------------------------------------------------

RatecodeID
1.0     2756472
NaN      540149
2.0       94420
99.0      41963
5.0       26501
3.0        8622
4.0        7092
6.0           7
Name: count, dtype: int64

------------------------------------------------------------

store_and_fwd_flag
N       2927431
None     540149
Y          7646
Name: count, dtype: int64

------------------------------------------------------------

payment_type
1    2444393
0     540149
2     390429
4      76481
3      23773
5          1
Name: count, dtype: int64
```

```
#Task 32
# Check the range of pickup and drop-off location IDs
print("PULocationID range:",main['PULocationID'].min(),"-", main['PULocationID'].max())
print("\n" + "---" * 20 + "\n")
print("DOLocationID range:",main['DOLocationID'].min(), "-", main['DOLocationID'].max())
```

```
PULocationID range: 1 - 265

------------------------------------------------------------

DOLocationID range: 1 - 265
```

```
# Task 33 - Investigate Suspicious Records
#Here we are only showing the first 10 records

# 1. Passenger count <= 0
print("1. Passenger Count <= 0")
print(main.loc[main['passenger_count'] <= 0,
```

```
              ['passenger_count', 'trip_distance']].head(10))

print("\n" + "---" * 20 + "\n")


# 2. Drop-off before pickup
print("2. Drop-off Before Pickup")
print(main.loc[main['tpep_dropoff_datetime'] < main['tpep_pickup_datetime'],
        ['tpep_pickup_datetime', 'tpep_dropoff_datetime']].head(10))

print("\n" + "---" * 20 + "\n")


# 3. Zero-duration trips
print("3. Zero-Duration Trips")
print(main.loc[main['tpep_dropoff_datetime'] == main['tpep_pickup_datetime'],
        ['tpep_pickup_datetime', 'tpep_dropoff_datetime', 'trip_distance']].head(10))

print("\n" + "---" * 20 + "\n")


# 4. Zero or negative trip distance
print("4. Zero or Negative Trip Distance")
print(main.loc[main['trip_distance'] <= 0,
        ['trip_distance', 'tpep_pickup_datetime', 'tpep_dropoff_datetime']].head(10))

print("\n" + "---" * 20 + "\n")


# 5. Extremely high trip distance
print("5. Extremely High Trip Distance")
print(main.loc[main['trip_distance'].nlargest(10).index,
 ['trip_distance', 'tpep_pickup_datetime', 'tpep_dropoff_datetime']])
```

```
2. Drop-off Before Pickup
        tpep_pickup_datetime tpep_dropoff_datetime
99620    2025-01-02 12:26:00   2025-01-02 11:29:58
459885   2025-01-06 16:00:00   2025-01-06 15:05:30
1315880  2025-01-15 15:00:00   2025-01-15 14:42:48
2029534  2025-01-23 01:44:59   2024-12-18 07:52:40
2058555  2025-01-23 12:30:00   2025-01-23 11:44:59
2660983  2025-01-29 14:00:00   2025-01-29 13:30:15
2945475  2025-01-01 10:01:27   2025-01-01 10:01:00
2945514  2025-01-01 10:01:53   2025-01-01 10:01:33
2951588  2025-01-02 08:01:41   2025-01-02 08:01:16
2952145  2025-01-02 11:01:37   2025-01-02 11:01:29

------------------------------------------------------------

3. Zero-Duration Trips
      tpep_pickup_datetime tpep_dropoff_datetime  trip_distance
92     2025-01-01 00:49:48   2025-01-01 00:49:48          0.00
11604  2025-01-01 01:42:36   2025-01-01 01:42:36          0.00
13820  2025-01-01 02:13:25   2025-01-01 02:13:25          0.00
14605  2025-01-01 02:09:52   2025-01-01 02:09:52          0.00
14607  2025-01-01 02:49:40   2025-01-01 02:49:40          0.00
16093  2025-01-01 02:21:20   2025-01-01 02:21:20          0.00
19599  2025-01-01 03:46:16   2025-01-01 03:46:16          0.00
31891  2025-01-01 12:44:07   2025-01-01 12:44:07          1.56
34038  2025-01-01 12:21:49   2025-01-01 12:21:49          0.00
34951  2025-01-01 12:53:45   2025-01-01 12:53:45          0.00

------------------------------------------------------------

4. Zero or Negative Trip Distance
      trip_distance tpep_pickup_datetime tpep_dropoff_datetime
92              0.0  2025-01-01 00:49:48   2025-01-01 00:49:48
204             0.0  2025-01-01 00:37:43   2025-01-01 00:37:53
```

```
3188501      276099.95  2025-01-19 14:51:00   2025-01-19 14:58:00
3240337      222167.49  2025-01-21 09:19:00   2025-01-21 09:45:00
3112173      206137.99  2025-01-16 10:47:00   2025-01-16 11:11:00
3374854      202771.63  2025-01-26 17:46:00   2025-01-26 17:59:00
3081597      189687.43  2025-01-14 19:18:00   2025-01-14 19:36:00
3069039      181139.99  2025-01-13 21:11:00   2025-01-13 21:17:00
3312563      168079.57  2025-01-24 08:33:00   2025-01-24 08:44:00
3195037      167452.94  2025-01-19 18:31:00   2025-01-19 18:58:00
3138510      164959.95  2025-01-17 14:25:00   2025-01-17 14:40:00
```

Phase B — Data Cleaning

```
clean_df=main.copy()
clean_df
```

	VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fw
0	1	2025-01-01 00:18:38	2025-01-01 00:26:59	1.0	1.60	1.0	
1	1	2025-01-01 00:32:40	2025-01-01 00:35:13	1.0	0.50	1.0	
2	1	2025-01-01 00:44:04	2025-01-01 00:46:01	1.0	0.60	1.0	
3	2	2025-01-01 00:14:27	2025-01-01 00:20:01	3.0	0.52	1.0	
4	2	2025-01-01 00:21:34	2025-01-01 00:25:06	3.0	0.66	1.0	
...	...	...	...	...	...	...	
3475221	2	2025-01-31 23:01:48	2025-01-31 23:16:29	NaN	3.35	NaN	
3475222	2	2025-01-31 23:50:29	2025-02-01 00:17:27	NaN	8.73	NaN	
3475223	2	2025-01-31 23:26:59	2025-01-31 23:43:01	NaN	2.64	NaN	
3475224	2	2025-01-31 23:14:34	2025-01-31 23:34:52	NaN	3.16	NaN	
3475225	2	2025-01-31 23:56:42	2025-02-01 00:07:27	NaN	2.29	NaN	

3475226 rows × 20 columns

```
#Task 34
print("Missing values in clean_df:",clean_df.isnull().sum())
```

```
Missing values in clean_df: VendorID                    0
tpep_pickup_datetime          0
tpep_dropoff_datetime         0
passenger_count          540149
trip_distance                 0
RatecodeID               540149
store_and_fwd_flag       540149
PULocationID                  0
DOLocationID                  0
payment_type                  0
fare_amount                   0
extra                         0
mta_tax                       0
tip_amount                    0
tolls_amount                  0
improvement_surcharge         0
total_amount                  0
congestion_surcharge     540149
Airport_fee              540149
cbd_congestion_fee            0
dtype: int64
```

```
#Task 35
duplicate_count = clean_df.duplicated().sum()

print("Duplicate records:",duplicate_count)

if duplicate_count > 0:
    print('Need to remove Duplicates')
    clean_df=clean_df.drop_duplicates()
    print("Duplicates removed.")
else:
    print("No duplicate found.")
```

```
Duplicate records: 0
No duplicate found.
```

```
#Task 36
#For pickup (dtype('<M8[us]')) into datetime64[us] we have used -->pd.to_datetime
clean_df['tpep_pickup_datetime'] = pd.to_datetime(clean_df['tpep_pickup_datetime'])


#For Dropoff into datetime64[us] we have used -->pd.to_datetime
clean_df['tpep_dropoff_datetime'] = pd.to_datetime(clean_df['tpep_dropoff_datetime'])

print(clean_df[['tpep_pickup_datetime', 'tpep_dropoff_datetime']].dtypes)
```

```
tpep_pickup_datetime     datetime64[us]
tpep_dropoff_datetime    datetime64[us]
dtype: object
```

```
#Task 37
invalid_passenger = clean_df[clean_df['passenger_count'] <= 0]

print("Invalid passenger-count records:", len(invalid_passenger))
print(invalid_passenger[['passenger_count','trip_distance','tpep_pickup_datetime','tpep_dropoff_datetime'

#Here we'll make them all as missing value[NaN]
#using np.nan
clean_df.loc[clean_df['passenger_count'] <= 0,'passenger_count'] = np.nan
```

```
Invalid passenger-count records: 24656
     passenger_count  trip_distance tpep_pickup_datetime tpep_dropoff_datetime
6                0.0            0.4  2025-01-01 00:14:47   2025-01-01 00:16:15
7                0.0            1.6  2025-01-01 00:39:27   2025-01-01 00:51:51
8                0.0            2.8  2025-01-01 00:53:43   2025-01-01 01:13:23
94               0.0            0.7  2025-01-01 00:11:27   2025-01-01 00:16:58
95               0.0            1.0  2025-01-01 00:19:30   2025-01-01 00:27:25
96               0.0            1.2  2025-01-01 00:33:13   2025-01-01 00:40:08
97               0.0            1.8  2025-01-01 00:45:05   2025-01-01 01:20:32
173              0.0            0.9  2025-01-01 00:01:44   2025-01-01 00:08:38
174              0.0            2.1  2025-01-01 00:14:06   2025-01-01 00:32:34
175              0.0            1.5  2025-01-01 00:52:53   2025-01-01 01:01:04
```

```
# Task 38
invalid_distance = clean_df[clean_df['trip_distance'] <= 0]

print("Zero/negative distance records:", len(invalid_distance))
print(invalid_distance[['trip_distance','tpep_pickup_datetime','tpep_dropoff_datetime','passenger_count']
```

```
Zero/negative distance records: 90893
      trip_distance tpep_pickup_datetime tpep_dropoff_datetime  \
92              0.0  2025-01-01 00:49:48   2025-01-01 00:49:48
204             0.0  2025-01-01 00:37:43   2025-01-01 00:37:53
358             0.0  2025-01-01 00:57:08   2025-01-01 00:57:16
505             0.0  2025-01-01 00:27:40   2025-01-01 00:59:30
619             0.0  2025-01-01 00:56:49   2025-01-01 00:56:54
706             0.0  2025-01-01 00:42:42   2025-01-01 00:42:44
1073            0.0  2025-01-01 00:43:22   2025-01-01 00:46:18
1345            0.0  2025-01-01 00:24:26   2025-01-01 00:24:34
1354            0.0  2025-01-01 00:14:57   2025-01-01 00:17:44
1496            0.0  2025-01-01 00:49:10   2025-01-01 00:49:14

      passenger_count
92                1.0
204               1.0
358               3.0
505               1.0
619               4.0
706               NaN
1073              1.0
1345              4.0
1354              1.0
1496              4.0
```

```
# Task 39
invalid_time = clean_df[clean_df['tpep_dropoff_datetime'] < clean_df['tpep_pickup_datetime']]

print("Drop-off before pickup records:", len(invalid_time))

print(invalid_time[
['tpep_pickup_datetime','tpep_dropoff_datetime','trip_distance']].head(10))


#This data will be removed as that is suspicous
clean_df = clean_df[clean_df['tpep_dropoff_datetime'] >=clean_df['tpep_pickup_datetime']].copy()
```

```
Drop-off before pickup records: 124
        tpep_pickup_datetime tpep_dropoff_datetime  trip_distance
99620    2025-01-02 12:26:00   2025-01-02 11:29:58          9.00
459885   2025-01-06 16:00:00   2025-01-06 15:05:30          3.80
1315880  2025-01-15 15:00:00   2025-01-15 14:42:48          1.00
2029534  2025-01-23 01:44:59   2024-12-18 07:52:40          3.10
2058555  2025-01-23 12:30:00   2025-01-23 11:44:59          3.90
2660983  2025-01-29 14:00:00   2025-01-29 13:30:15          1.50
2945475  2025-01-01 10:01:27   2025-01-01 10:01:00         19.19
2945514  2025-01-01 10:01:53   2025-01-01 10:01:33         12.17
2951588  2025-01-02 08:01:41   2025-01-02 08:01:16         14.89
2952145  2025-01-02 11:01:37   2025-01-02 11:01:29         12.71
```

```
# Task 40
clean_df['trip_duration_minutes'] = ((clean_df['tpep_dropoff_datetime'] -clean_df['tpep_pickup_datetime']

print(clean_df['trip_duration_minutes'].describe())
print('-'*200)

#Showing lonmgest trip durations
print("\nLongest trip durations:")
print(clean_df.nlargest(10,'trip_duration_minutes')[
        ['tpep_pickup_datetime',
         'tpep_dropoff_datetime',
         'trip_duration_minutes',
         'trip_distance']])
```

```
count    3.475102e+06
mean     1.503353e+01
std      2.712825e+01
min      0.000000e+00
25%      7.283333e+00
50%      1.170000e+01
75%      1.833333e+01
max      5.626317e+03
Name: trip_duration_minutes, dtype: float64
----------------------------------------------------------------------------------------------------------------------------------------

Longest trip durations:
        tpep_pickup_datetime tpep_dropoff_datetime  trip_duration_minutes  \
1858816  2025-01-21 12:23:41   2025-01-25 10:10:00            5626.316667
1858817  2025-01-21 12:23:41   2025-01-25 10:10:00            5626.316667
414374   2025-01-06 02:12:11   2025-01-09 13:54:19            5022.133333
2460536  2025-01-27 11:58:55   2025-01-30 09:42:55            4184.000000
898201   2025-01-11 08:04:00   2025-01-13 16:49:28            3405.466667
2147536  2025-01-24 07:07:21   2025-01-26 10:09:44            3062.383333
85681    2025-01-02 09:40:31   2025-01-04 09:37:01            2876.500000
1536147  2025-01-17 16:45:49   2025-01-19 16:40:59            2875.166667
556772   2025-01-07 18:07:33   2025-01-09 16:12:05            2764.533333
3417048  2025-01-29 21:39:23   2025-01-31 16:48:43            2589.333333

         trip_distance
1858816           6.55
1858817           6.55
414374           91.51
2460536           0.00
898201            0.04
2147536           0.00
85681             0.00
1536147           1.98
556772            1.86
3417048           7.52
```

Task 41 - Cleaning Decisions

- Created a separate copy of the original dataset as `clean_df`.
- No duplicate records were found.
- Passenger counts less than or equal to zero were identified as suspicious and converted to missing values (`NaN`).
- Zero and negative trip distances were investigated but not automatically removed.
- Records where drop-off occurred before pickup were identified as suspicious and removed.
- Extremely long trip durations were investigated as potential data-quality issues.

Feature Engineering

```
# Feature 1 - Trip Duration
clean_df['trip_duration_minutes'] = (
    (clean_df['tpep_dropoff_datetime'] -clean_df['tpep_pickup_datetime'])
    .dt.total_seconds() / 60)

print(clean_df[['tpep_pickup_datetime',
                'tpep_dropoff_datetime',
                'trip_duration_minutes']].head())
```

```
  tpep_pickup_datetime tpep_dropoff_datetime  trip_duration_minutes
0  2025-01-01 00:18:38   2025-01-01 00:26:59               8.350000
1  2025-01-01 00:32:40   2025-01-01 00:35:13               2.550000
2  2025-01-01 00:44:04   2025-01-01 00:46:01               1.950000
3  2025-01-01 00:14:27   2025-01-01 00:20:01               5.566667
4  2025-01-01 00:21:34   2025-01-01 00:25:06               3.533333
```

```
# Feature 2 - Pickup Date

clean_df['pickup_date'] = clean_df['tpep_pickup_datetime'].dt.date
clean_df[['tpep_pickup_datetime', 'pickup_date']].head()
```

	tpep_pickup_datetime	pickup_date
0	2025-01-01 00:18:38	2025-01-01
1	2025-01-01 00:32:40	2025-01-01
2	2025-01-01 00:44:04	2025-01-01
3	2025-01-01 00:14:27	2025-01-01
4	2025-01-01 00:21:34	2025-01-01

```
# Feature 3 - Pickup Hour

clean_df['pickup_hour'] = (clean_df['tpep_pickup_datetime'].dt.hour)

print(clean_df[['tpep_pickup_datetime',
                'pickup_hour']].head())
```

```
  tpep_pickup_datetime  pickup_hour
0  2025-01-01 00:18:38            0
1  2025-01-01 00:32:40            0
2  2025-01-01 00:44:04            0
3  2025-01-01 00:14:27            0
4  2025-01-01 00:21:34            0
```

```
# Feature 4 - Day of Week

clean_df['day_of_week'] = (clean_df['tpep_pickup_datetime'].dt.day_name())

print(clean_df[['tpep_pickup_datetime',
                'day_of_week']].head(10))
```

```
  tpep_pickup_datetime day_of_week
0  2025-01-01 00:18:38   Wednesday
1  2025-01-01 00:32:40   Wednesday
2  2025-01-01 00:44:04   Wednesday
3  2025-01-01 00:14:27   Wednesday
4  2025-01-01 00:21:34   Wednesday
5  2025-01-01 00:48:24   Wednesday
6  2025-01-01 00:14:47   Wednesday
7  2025-01-01 00:39:27   Wednesday
8  2025-01-01 00:53:43   Wednesday
9  2025-01-01 00:00:02   Wednesday
```

```
# Feature 5 - Weekend or WeekDay

clean_df['is_weekend'] = (clean_df['tpep_pickup_datetime'].dt.dayofweek >= 5)

print(clean_df[['day_of_week',
                'is_weekend']].head(10))
```

```
  day_of_week  is_weekend
0   Wednesday       False
1   Wednesday       False
2   Wednesday       False
3   Wednesday       False
4   Wednesday       False
5   Wednesday       False
6   Wednesday       False
```

```
7   Wednesday       False
8   Wednesday       False
9   Wednesday       False
```

```
# Feature 6 - Average Speed
#Average Speed = Trip Distance / Trip Duration in hours
clean_df['average_speed_mph'] = np.where(clean_df['trip_duration_minutes'] > 0,
    clean_df['trip_distance'] /(clean_df['trip_duration_minutes'] / 60),np.nan)

print(clean_df[['trip_distance',
                'trip_duration_minutes',
                'average_speed_mph']].head())
```

```
   trip_distance  trip_duration_minutes  average_speed_mph
0           1.60               8.350000          11.497006
1           0.50               2.550000          11.764706
2           0.60               1.950000          18.461538
3           0.52               5.566667           5.604790
4           0.66               3.533333          11.207547
```

```
# Feature 7 - Distance Category

clean_df['distance_category'] = pd.cut(
    clean_df['trip_distance'],
    bins=[-np.inf, 3, 10, np.inf],
    labels=['Short', 'Medium', 'Long']
)

print(clean_df[['trip_distance',
                'distance_category']].head())

print('This gives distance groups for comparison and visualization')
```

```
   trip_distance distance_category
0           1.60             Short
1           0.50             Short
2           0.60             Short
3           0.52             Short
4           0.66             Short
This gives distance groups for comparison and visualization
```

```
# Feature 8 - Duration Category
# Very Short: <= 5 minutes
# Short: 5-15 minutes
# Medium: 15-30 minutes
# Long: > 30 minutes
clean_df['duration_category'] = pd.cut(
    clean_df['trip_duration_minutes'],
    bins=[-np.inf, 5, 15, 30, np.inf],
    labels=['Very Short', 'Short', 'Medium', 'Long']
)

print(clean_df[['trip_duration_minutes',
                'duration_category']].head())
```

```
   trip_duration_minutes duration_category
0               8.350000             Short
1               2.550000        Very Short
2               1.950000        Very Short
3               5.566667             Short
4               3.533333        Very Short
```

```
# Feature 9 - Route

clean_df['route'] = (
    clean_df['PULocationID'].astype(str)
    + ' → ' +
    clean_df['DOLocationID'].astype(str)
)

print(clean_df[['PULocationID',
                'DOLocationID',
                'route']].head())
```

```
   PULocationID  DOLocationID      route
0           229           237  229 → 237
1           236           237  236 → 237
2           141           141  141 → 141
```

```
3           244           244  244 → 244
4           244           116  244 → 116
```

Feature Engineering Summary

Created useful features for **trip duration, pickup time, weekdays/weekends, average speed, distance, duration, and routes** to support further analysis.

Phase C — Univariate Analysis

- What does each individual variable look like?

Trip Distance

```
#1 Trip Distance

print("Mean:", clean_df['trip_distance'].mean())
print("Median:", clean_df['trip_distance'].median())
print("Minimum:", clean_df['trip_distance'].min())
print("Maximum:", clean_df['trip_distance'].max())
print("Std Dev:", clean_df['trip_distance'].std())
```

```
Mean: 5.855109182406734
Median: 1.67
Minimum: 0.0
Maximum: 276423.57
Std Dev: 564.6116718928984
```

```
# Percentiles

print("25th Percentile:", clean_df['trip_distance'].quantile(0.25))
print("50th Percentile:", clean_df['trip_distance'].quantile(0.50))
print("75th Percentile:", clean_df['trip_distance'].quantile(0.75))
print("90th Percentile:", clean_df['trip_distance'].quantile(0.90))
```

```
25th Percentile: 0.98
50th Percentile: 1.67
75th Percentile: 3.1
90th Percentile: 7.7
```

```
#IQR Outlier
Q1 = clean_df['trip_distance'].quantile(0.25)
Q3 = clean_df['trip_distance'].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

outliers = clean_df[(clean_df['trip_distance'] < lower_bound) | (clean_df['trip_distance'] > upper_bound)]

print("Number of potential outliers:", len(outliers))
```

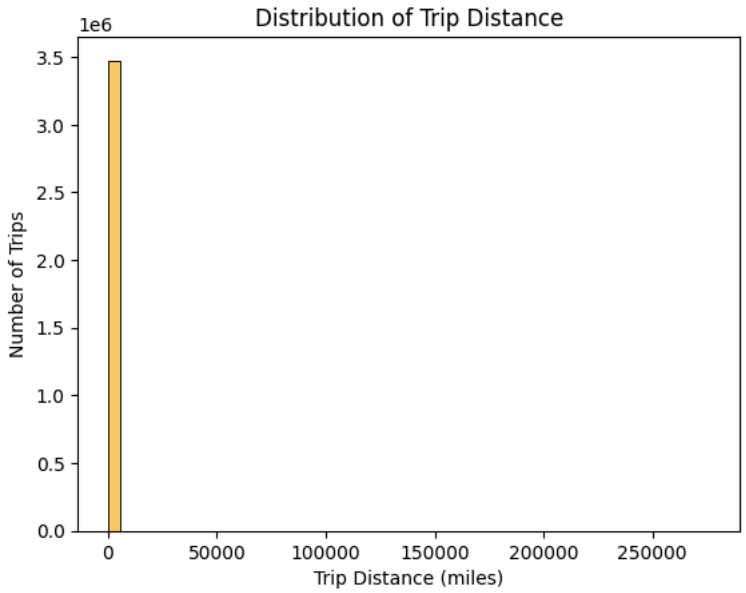
```
Q1: 0.98
Q3: 3.1
IQR: 2.12
Lower Bound: -2.2
Upper Bound: 6.28
Number of potential outliers: 422384
```

Trip Duration

```
# Trip Distance - Distribution
sns.histplot(clean_df['trip_distance'],bins=50,color='#F7B731')

plt.title("Distribution of Trip Distance")
plt.xlabel("Trip Distance (miles)")
plt.ylabel("Number of Trips")

plt.show()
```




```
print("Trip Distance Boxplot")

plt.figure(figsize=(10, 5))

sns.boxplot(
    x=clean_df['trip_distance'],
    color='#F7B731'
)

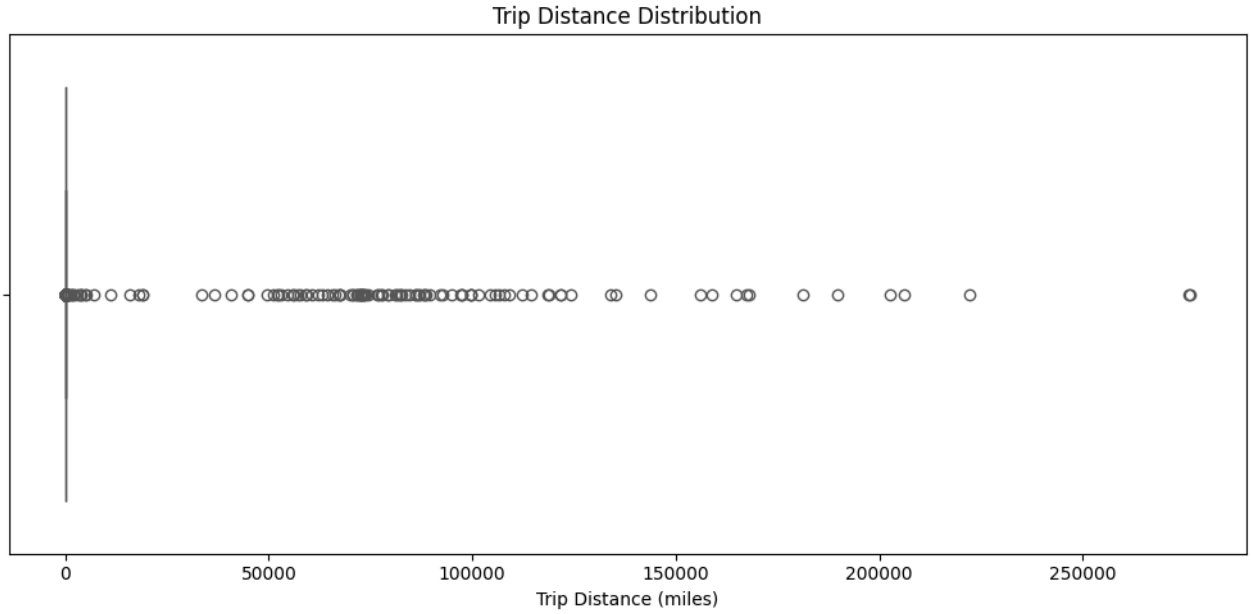
plt.title('Trip Distance Distribution')
plt.xlabel('Trip Distance (miles)')

plt.tight_layout()
plt.show()

print('---' * 20)

print("Insight:")
print("Most trips are concentrated at lower distances.")
print("Several extreme values indicate potential distance outliers.")
```

```
Trip Distance Boxplot
```



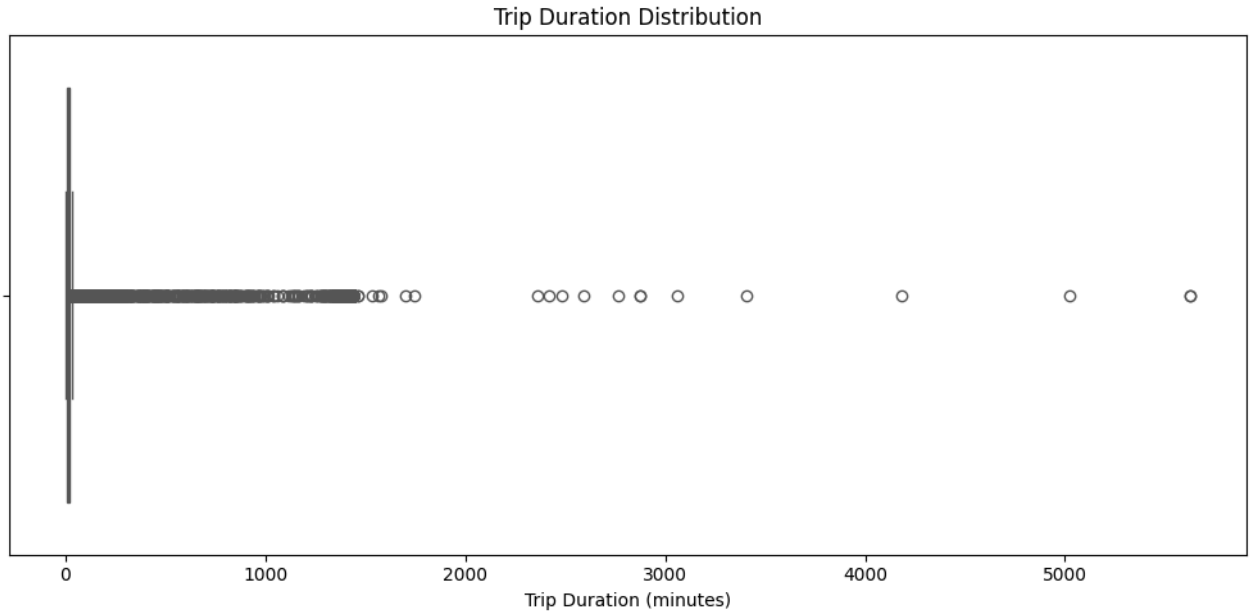

```
------------------------------------------------------------
Insight:
Most trips are concentrated at lower distances.
Several extreme values indicate potential distance outliers.
```

```
plt.figure(figsize=(10, 5))

sns.boxplot(x=clean_df['trip_duration_minutes'],color='#F7B731')

plt.title('Trip Duration Distribution')
plt.xlabel('Trip Duration (minutes)')

plt.tight_layout()
plt.show()
print('---'*20)
print("Insight:")
print("Most trip durations are concentrated in a lower range.")
print("Several extreme values indicate potential duration outliers.")
```




```
------------------------------------------------------------
Insight:
Most trip durations are concentrated in a lower range.
Several extreme values indicate potential duration outliers.
```

```
# Trip Duration
print("Mean of Trip Duration:", clean_df['trip_duration_minutes'].mean())
print("Median of Trip Duration:", clean_df['trip_duration_minutes'].median())
```

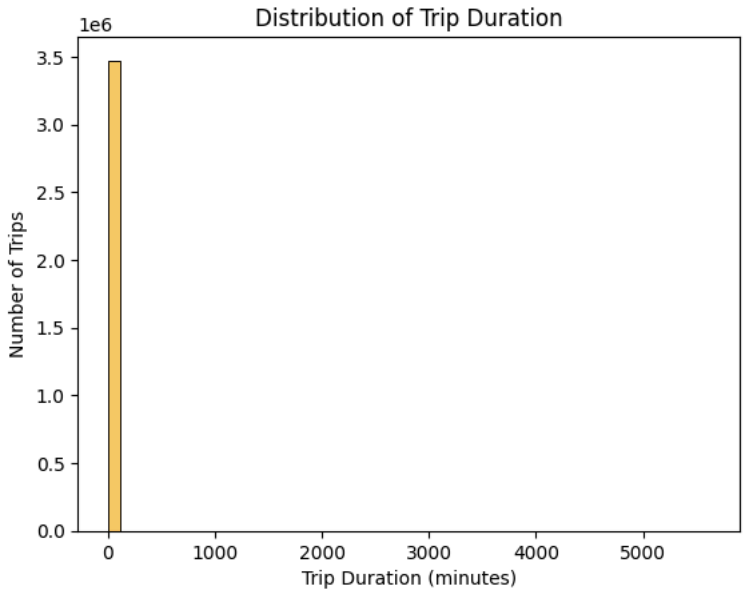
```
print("Minimum of Trip Duration:", clean_df['trip_duration_minutes'].min())
print("Maximum of Trip Duration:", clean_df['trip_duration_minutes'].max())
```

```
Mean of Trip Duration: 15.033534301055512
Median of Trip Duration: 11.7
Minimum of Trip Duration: 0.0
Maximum of Trip Duration: 5626.316666666667
```

```
sns.histplot(clean_df['trip_duration_minutes'],bins=50,color='#F7B731')

plt.title("Distribution of Trip Duration")
plt.xlabel("Trip Duration (minutes)")
plt.ylabel("Number of Trips")

plt.show()
```

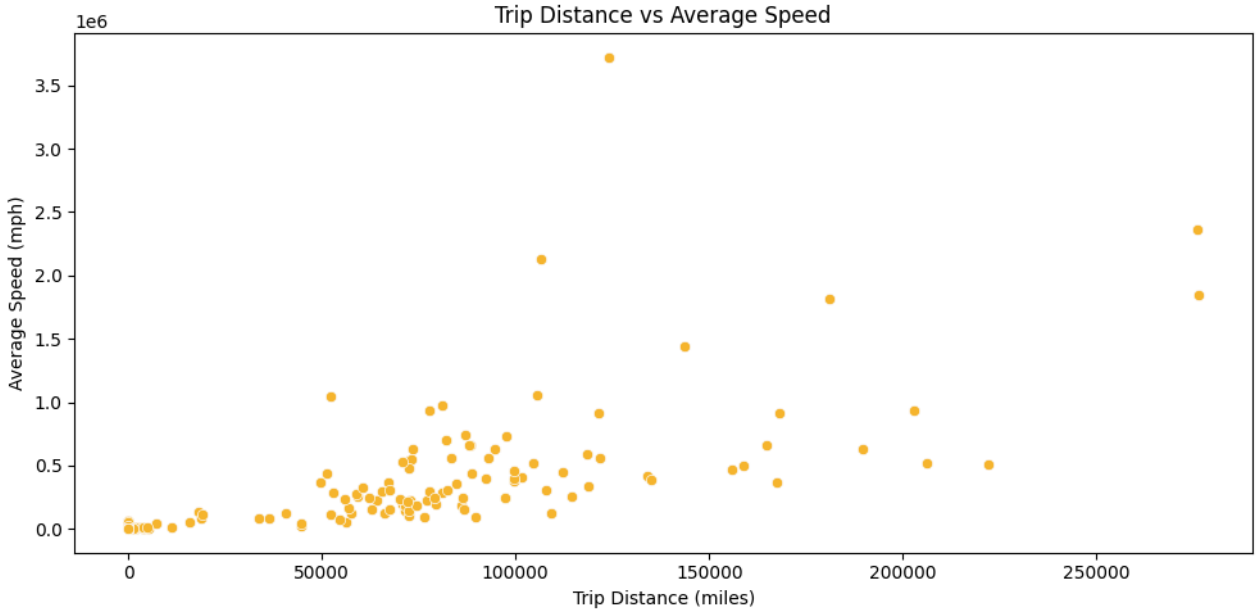



```
plt.figure(figsize=(10, 5))

sns.scatterplot(x=clean_df['trip_distance'],y=clean_df['average_speed_mph'],color='#F7B731')

plt.title('Trip Distance vs Average Speed')
plt.xlabel('Trip Distance (miles)')
plt.ylabel('Average Speed (mph)')

plt.tight_layout()
plt.show()
print('---'*20)
print("Insight:")
print("Trip distance and average speed show a strong positive relationship.")
print("Some extreme values indicate potential data-quality issues.")
```




```
------------------------------------------------------------
Insight:
Trip distance and average speed show a strong positive relationship.
Some extreme values indicate potential data-quality issues.
```

```
#IQR Outlier

Q1 = clean_df['trip_duration_minutes'].quantile(0.25)
Q3 = clean_df['trip_duration_minutes'].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

outliers = clean_df[
    (clean_df['trip_duration_minutes'] < lower_bound) |
    (clean_df['trip_duration_minutes'] > upper_bound)
]

print("Potential outliers:", len(outliers))
```

```
Q1: 7.283333333333333
Q3: 18.333333333333332
IQR: 11.049999999999999
Lower Bound: -9.291666666666666
Upper Bound: 34.90833333333333
Potential outliers: 191020
```

Passenger Count

```
print("Q. What is the most common passenger count?")
print(clean_df['passenger_count'].value_counts(dropna=False))
```

```
Q. What is the most common passenger count?
passenger_count
1.0    2322429
NaN     564686
2.0     407761
3.0      91409
4.0      59009
5.0      17786
6.0      12004
8.0         11
7.0          4
9.0          3
Name: count, dtype: int64
```

```
print('Q. Are there unusual passenger counts?')
print("Most common passenger count:",clean_df['passenger_count'].mode()[0])

print('\nNote--> Since we specifically want the most common passenger count, mode is the appropriate meas
```

```
Q. Are there unusual passenger counts?
Most common passenger count: 1.0

Note--> Since we specifically want the most common passenger count, mode is the appropriate measure.
```

```
print('Q. How frequently are records missing passenger-count information ?')

missing_passengers = clean_df['passenger_count'].isnull().sum()

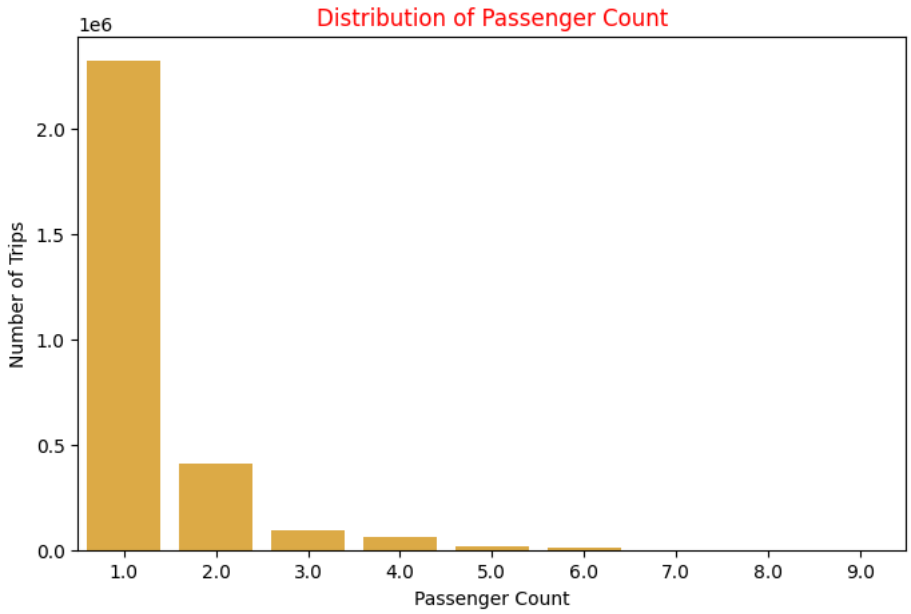
print("Missing passenger-count records:", missing_passengers)
print( "Missing percentage:",(missing_passengers / len(clean_df)) * 100)
```

```
Q. How frequently are records missing passenger-count information ?
Missing passenger-count records: 564686
Missing percentage: 16.249479871382192
```

```
# Passenger Count - Count Plot
plt.figure(figsize=(8, 5))
sns.countplot(data=clean_df,x='passenger_count',color='#F7B731')

plt.title("Distribution of Passenger Count",color='red')
plt.xlabel("Passenger Count")
plt.ylabel("Number of Trips")

plt.show()
```



Pickup Hour

```
print("Q. Which hour has the highest trip activity?")

hourly_trips = clean_df['pickup_hour'].value_counts()

print("Busiest hour:", hourly_trips.idxmax())
print("Number of trips:", hourly_trips.max())
```

```
Q. Which hour has the highest trip activity?
Busiest hour: 18
Number of trips: 267949
```

```
print("Q. Which hour has the lowest trip activity?")

print("Least busy hour:", hourly_trips.idxmin())
#idxmin() gives the hour with the lowest number of trips.It shows the place of higest valeue
```

```
print("Number of trips:", hourly_trips.min())
```

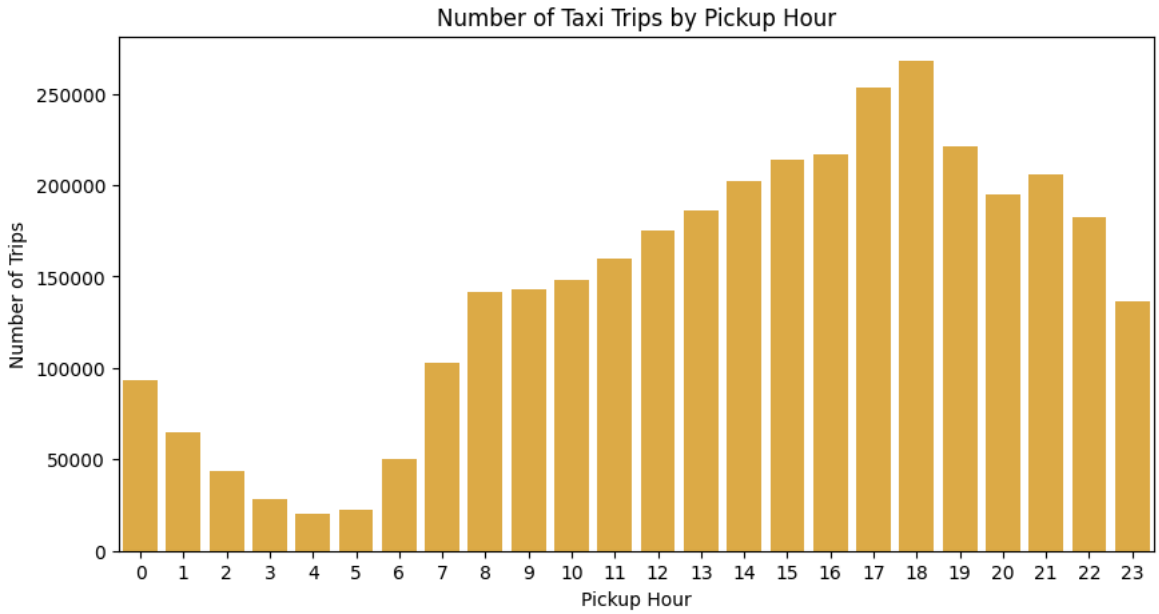
```
Q. Which hour has the lowest trip activity?
Least busy hour: 4
Number of trips: 20031
```

```
print("Q. Is demand evenly distributed throughout the day?")
print("Ans--> \nWe can see in this graph that the start is good but 3-4-5 has gone till bottom then it ge

#Figyure to get the info
plt.figure(figsize=(10, 5))
sns.barplot(x=hourly_trips.index, y=hourly_trips.values,color='#F7B731')

plt.title("Number of Taxi Trips by Pickup Hour")
plt.xlabel("Pickup Hour")
plt.ylabel("Number of Trips")
plt.show()
```

```
Q. Is demand evenly distributed throughout the day?
Ans-->
We can see in this graph that the start is good but 3-4-5 has gone till bottom then it gets a high push till 18th hour
```



Day of Week

```
day_order = [
    'Monday', 'Tuesday', 'Wednesday',
    'Thursday', 'Friday', 'Saturday', 'Sunday'
]

daily_trips = clean_df['day_of_week'].value_counts().reindex(day_order)
print("Number of trips on each day of the week:")
print(daily_trips)
```

```
Number of trips on each day of the week:
day_of_week
Monday       363755
Tuesday      453966
Wednesday    577341
Thursday     614208
Friday       586853
Saturday     476393
Sunday       402586
Name: count, dtype: int64
```

```
print("Which day has the highest trip activity?")
print("Busiest day:", daily_trips.idxmax())
print("Number of trips:", daily_trips.max())

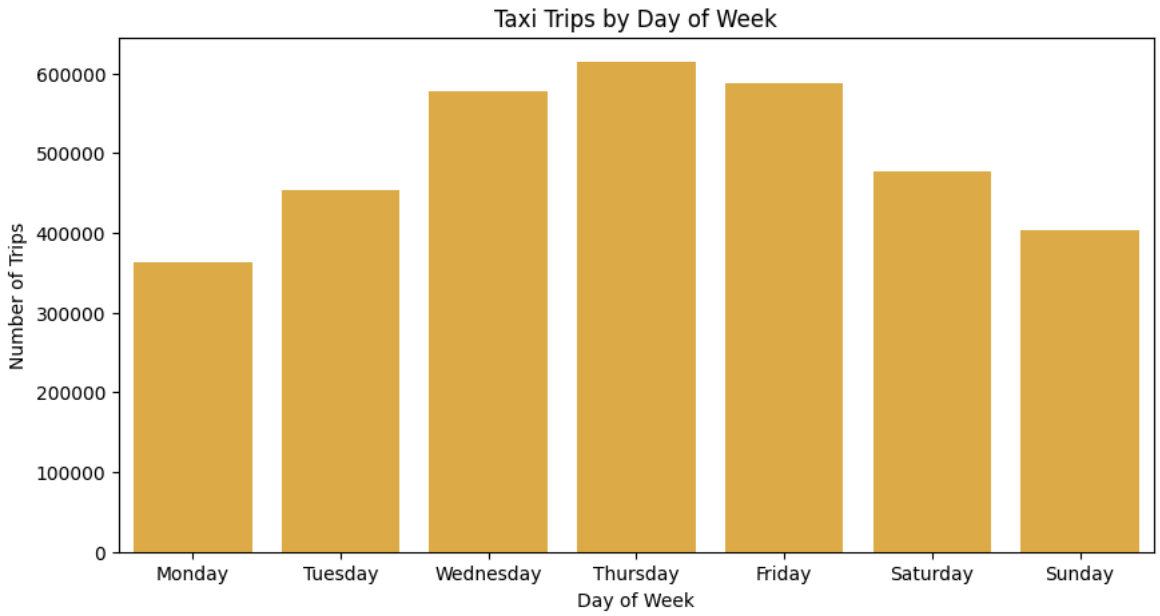
print("\nWhich day has the lowest trip activity?")
print("Least busy day:", daily_trips.idxmin())
print("Number of trips:", daily_trips.min())
```

```
Which day has the highest trip activity?
Busiest day: Thursday
Number of trips: 614208

Which day has the lowest trip activity?
Least busy day: Monday
Number of trips: 363755
```

```
plt.figure(figsize=(10, 5))
sns.barplot(x=daily_trips.index,y=daily_trips.values,color='#F7B731')
plt.title("Taxi Trips by Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Number of Trips")

plt.show()
print('Insight--> \n Thursday has the highest number and monday has lowest \n Trip activity is higher on
```




```
Insight-->
 Thursday has the highest number and monday has lowest
 Trip activity is higher on wekdays
```

Payment Type

```
print("How many trips belong to each payment type?")

payment_trips = clean_df['payment_type'].value_counts()

payment_names = {
    0: 'Flex Fare',
    1: 'Credit Card',
    2: 'Cash',
    3: 'No Charge',
    4: 'Dispute',
    5: 'Unknown',
    6: 'Voided Trip'
}

payment_trips_named = payment_trips.rename(index=payment_names)

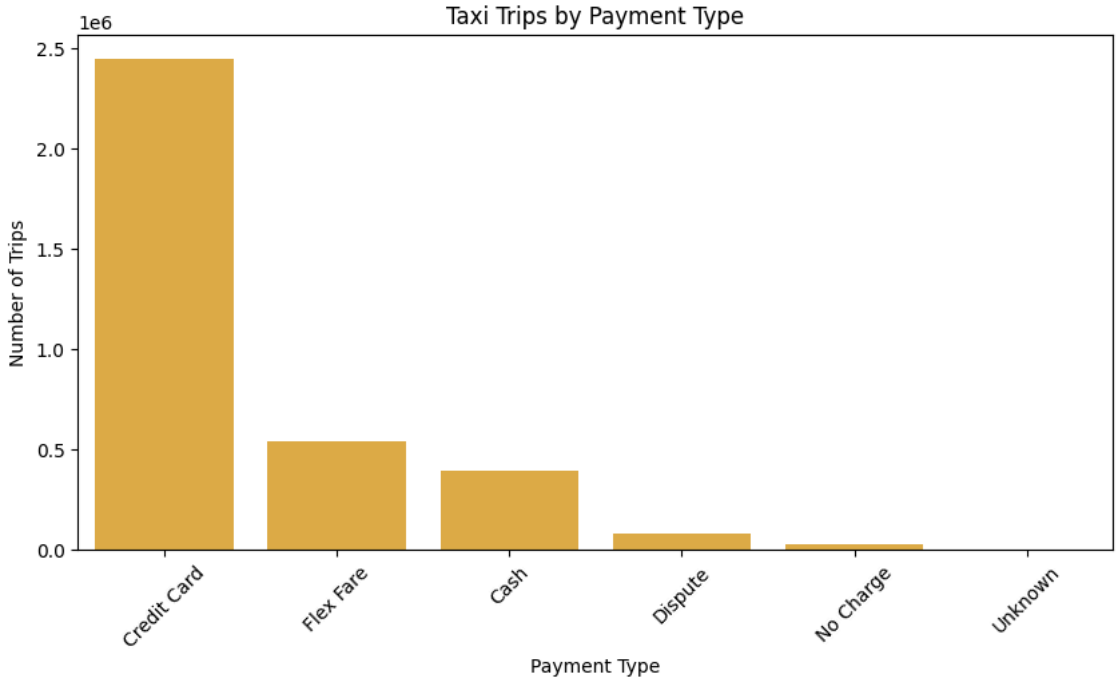
print(payment_trips_named)  #used
```

```
How many trips belong to each payment type?
payment_type
Credit Card    2444387
Flex Fare       540031
Cash            390429
Dispute          76481
No Charge        23773
Unknown              1
Name: count, dtype: int64
```

```
plt.figure(figsize=(10, 5))
```

```
sns.barplot(x=payment_trips_named.index,y=payment_trips_named.values,color='#F7B731')
plt.title("Taxi Trips by Payment Type")
plt.xlabel("Payment Type")
plt.ylabel("Number of Trips")
plt.xticks(rotation=45)

plt.show()
print('Insights-->\n Payment type Credit card is clearly the most common.\n Others are somewhat lower as c
```




```
Insights-->
 Payment type Credit card is clearly the most common.
 Others are somewhat lower as compared to Credit Card
```

Vendor

```
vendor_trips = clean_df['VendorID'].value_counts()

print("How many trips are recorded by each vendor?")
print(vendor_trips)
```

```
How many trips are recorded by each vendor?
VendorID
2    2719860
1     753665
7       1206
6        371
Name: count, dtype: int64
```

```
print("Which vendor has the highest number of trips?")
print("Most common vendor:", vendor_trips.idxmax())
print("Number of trips:", vendor_trips.max())
```

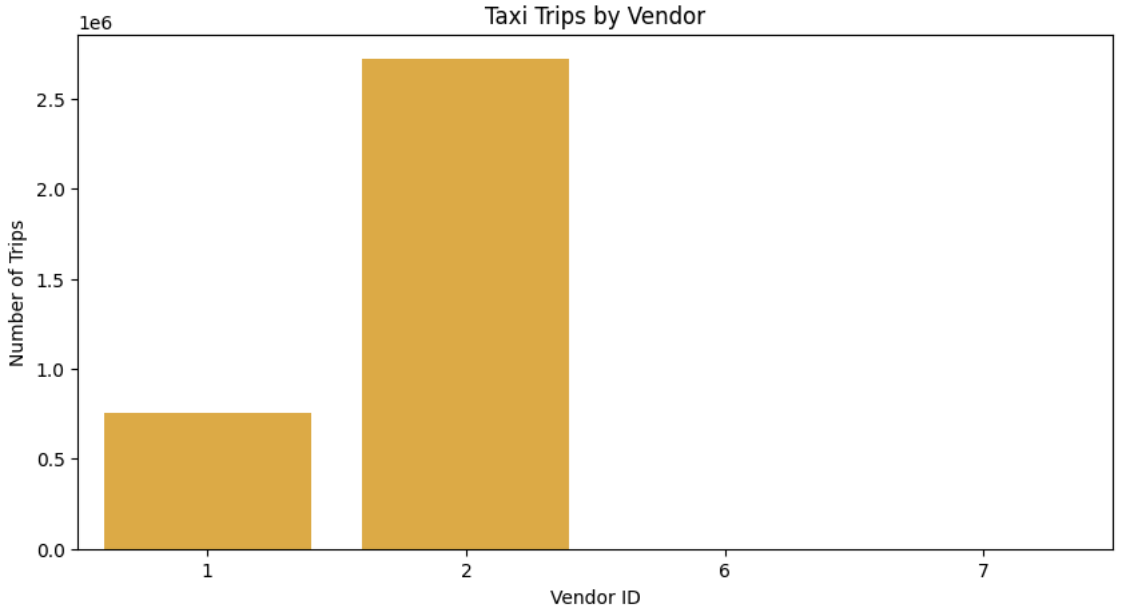
```
Which vendor has the highest number of trips?
Most common vendor: 2
Number of trips: 2719860
```

```
plt.figure(figsize=(10, 5))

sns.barplot(x=vendor_trips.index,y=vendor_trips.values,color='#F7B731')

plt.title("Taxi Trips by Vendor")
plt.xlabel("Vendor ID")
plt.ylabel("Number of Trips")

plt.show()
print('Insight-->\n Vendor 2 recorded the highest number of trips, followed by Vendor 1.\n Vendors 6 and
```




```
Insight-->
 Vendor 2 recorded the highest number of trips, followed by Vendor 1.
 Vendors 6 and 7 account for very few trips in comparison.
```

Rate code

```
ratecode_trips = clean_df['RatecodeID'].value_counts()

print("How many trips belong to each rate code?\n")
print(ratecode_trips)
```

```
How many trips belong to each rate code?

RatecodeID
1.0     2756471
2.0       94420
99.0      41958
5.0       26501
3.0        8622
4.0        7092
6.0           7
Name: count, dtype: int64
```

```
print("Which rate code is most frequently used?")
print("Most common rate code:", ratecode_trips.idxmax())
print("Number of trips:", ratecode_trips.max())
```

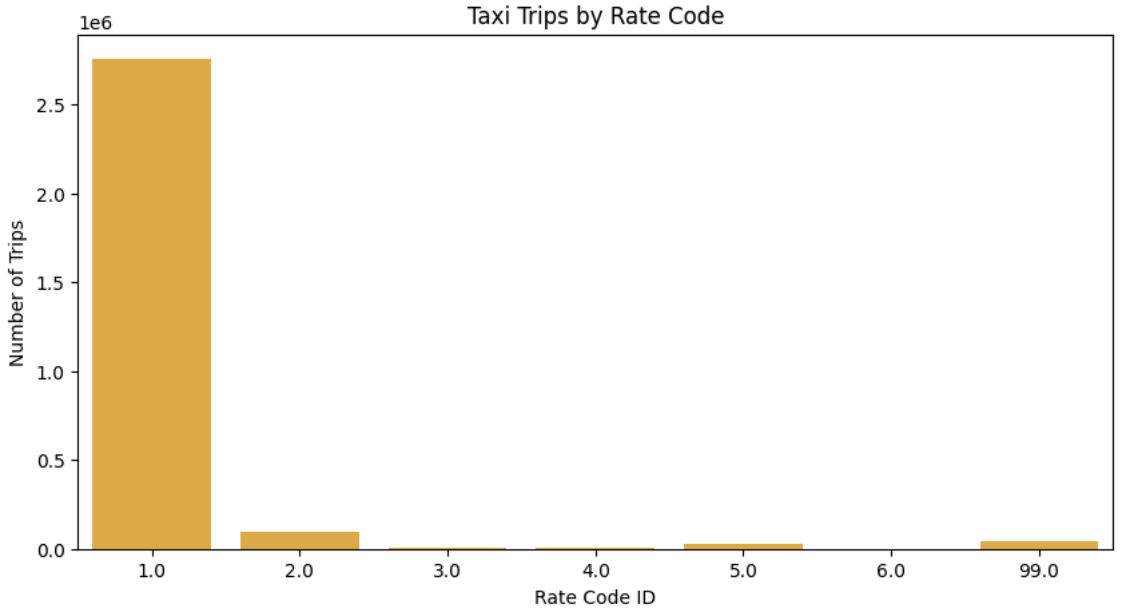
```
Which rate code is most frequently used?
Most common rate code: 1.0
Number of trips: 2756471
```

```
plt.figure(figsize=(10, 5))

sns.barplot(x=ratecode_trips.index,y=ratecode_trips.values,color='#F7B731')

plt.title("Taxi Trips by Rate Code")
plt.xlabel("Rate Code ID")
plt.ylabel("Number of Trips")

plt.show()
print('Insight-->\n Rate code 1 is highest used rate code.\n Others are far behind Rate code 1')
```




```
Insight-->
 Rate code 1 is highest used rate code.
 Others are far behind Rate code 1
```

Phase D - Time-Based Analysis

```
print("Q.42 Which hour has the highest number of trips?")
print("Busiest hour:", hourly_trips.idxmax())
print("Number of trips:", hourly_trips.max())
```

```
Q.42 Which hour has the highest number of trips?
Busiest hour: 18
Number of trips: 267949
```

```
print("Q.43 Which hour has the lowest number of trips?")
print("Least busy hour:",hourly_trips.idxmin())
print("Number of trips:",hourly_trips.min())
```

```
Q.43 Which hour has the lowest number of trips?
Least busy hour: 4
Number of trips: 20031
```

```
print("Q.44 Which day of the week has the highest number of trips?")
print("Busiest day:", daily_trips.idxmax())
print("Number of trips:", daily_trips.max())
```

```
Q.44 Which day of the week has the highest number of trips?
Busiest day: Thursday
Number of trips: 614208
```

```
print("Q.45 Which day of the week has the lowest number of trips?")
print("Least busy day:", daily_trips.idxmin())
print("Number of trips:", daily_trips.min())
```

```
Q.45 Which day of the week has the lowest number of trips?
Least busy day: Monday
Number of trips: 363755
```

```
print("Q.46  How does weekday activity compare with weekend activity?")
weekday_trips = clean_df[clean_df['is_weekend'] == False].shape[0] #is_weekend is used to separate trips
weekend_trips = clean_df[clean_df['is_weekend'] == True].shape[0]

print("\nWeekday trips:", weekday_trips)
print("\nWeekend trips:", weekend_trips)
```

```
Q.46  How does weekday activity compare with weekend activity?

Weekday trips: 2596123

Weekend trips: 878979
```

```
print("Q.47 Which hours have the longest average trip duration?\n")

avg_duration_by_hour = clean_df.groupby('pickup_hour')['trip_duration_minutes'].mean()

print(avg_duration_by_hour.sort_values(ascending=False).head())
```

```
Q.47 Which hours have the longest average trip duration?

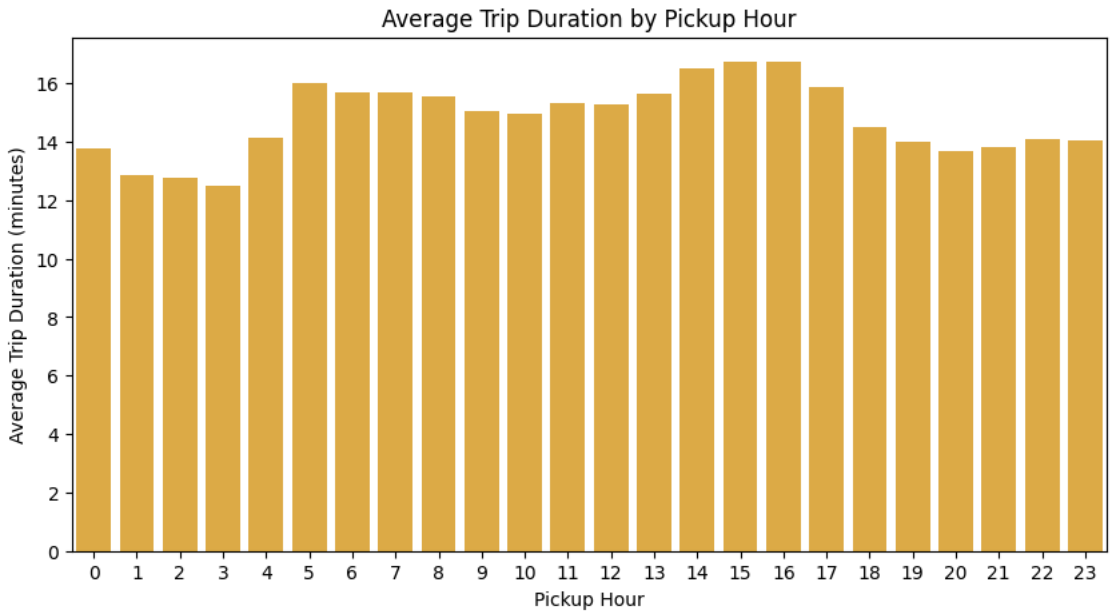
pickup_hour
16    16.739213
15    16.725188
14    16.515018
5     15.999516
17    15.857250
Name: trip_duration_minutes, dtype: float64
```

```
plt.figure(figsize=(10, 5))

sns.barplot(
    x=avg_duration_by_hour.index,
    y=avg_duration_by_hour.values,
    color='#F7B731'
)

plt.title("Average Trip Duration by Pickup Hour")
plt.xlabel("Pickup Hour")
plt.ylabel("Average Trip Duration (minutes)")

plt.show()
print('---'*20)
print("Insight-->")
print("Average trip duration changes across the day.")
print("The highest average duration is around 4 PM.")
```




```
------------------------------------------------------------
Insight-->
Average trip duration changes across the day.
The highest average duration is around 4 PM.
```

```
avg_distance_by_hour = clean_df.groupby('pickup_hour')['trip_distance'].mean()

print("Q.48 Which hours have the highest average trip distance?")
print(avg_distance_by_hour.sort_values(ascending=False).head())
```

```
Q.48 Which hours have the highest average trip distance?
pickup_hour
5    39.514377
6    21.422510
8    10.097616
4     9.157623
7     8.145909
Name: trip_distance, dtype: float64
```

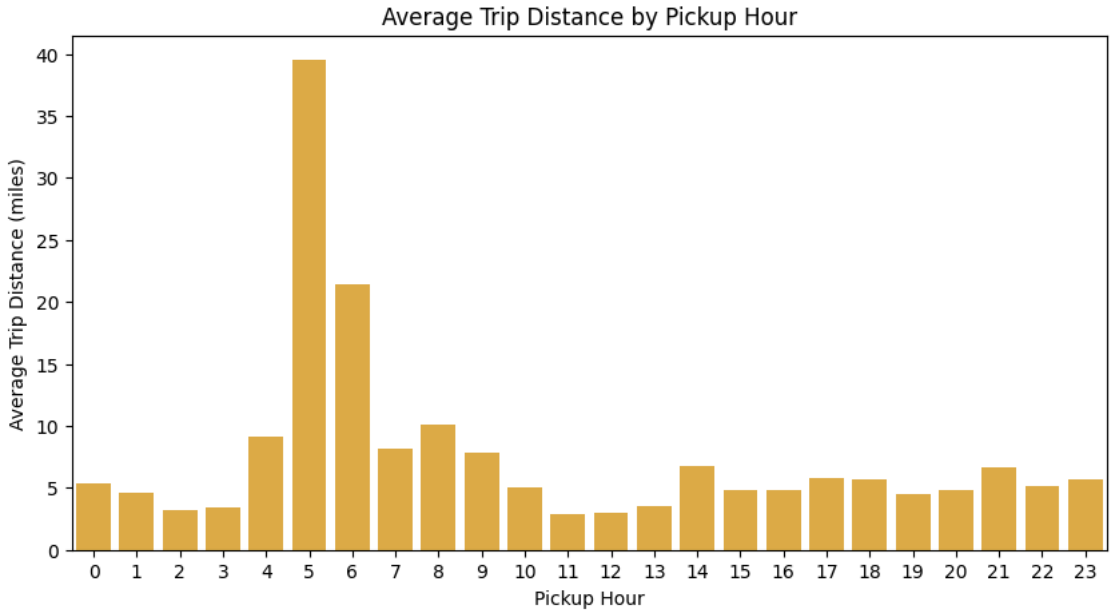
```
plt.figure(figsize=(10, 5))
```

```
sns.barplot(
    x=avg_distance_by_hour.index,
    y=avg_distance_by_hour.values,
    color='#F7B731'
)

plt.title("Average Trip Distance by Pickup Hour")
plt.xlabel("Pickup Hour")
plt.ylabel("Average Trip Distance (miles)")

plt.show()

print("Insight:")
print("Average trip distance changes across the day.")
print("The highest average distance occurs during the early morning hours.")
```




```
Insight:
Average trip distance changes across the day.
The highest average distance occurs during the early morning hours.
```

```
avg_speed_by_hour = clean_df.groupby('pickup_hour')['average_speed_mph'].mean()

print("Q.49 Does average speed change throughout the day?")
print(avg_speed_by_hour)
```

```
Q.49 Does average speed change throughout the day?
pickup_hour
0      24.196205
1      24.508146
2      14.327115
3      14.899004
4      27.525414
5     401.939894
6     110.895544
7      27.017546
8      42.228761
9      28.164030
10     18.486738
11     10.556516
12     10.905897
13     16.894598
14     29.114647
15     17.442057
16     19.648574
17     21.443286
18     18.387251
19     16.588495
20     19.017172
21     40.168101
22     20.949505
23     24.949758
Name: average_speed_mph, dtype: float64
```

Phase E — Trip Distance & Duration Analysis

```
print("Q.50 What is the average trip distance?")
print("Average trip distance:", clean_df['trip_distance'].mean())
```

```
Q.50 What is the average trip distance?
Average trip distance: 5.855109182406734
```

```
print("Q.51 What is the median trip distance?")
print("Median trip distance:", clean_df['trip_distance'].median())
```

```
Q.51 What is the median trip distance?
Median trip distance: 1.67
```

```
print("Q.52 What is the average trip duration?")
print("Average trip duration:", clean_df['trip_duration_minutes'].mean())
```

```
Q.52 What is the average trip duration?
Average trip duration: 15.033534301055512
```

```
print("Q.53 What is the median trip duration?")
print("Median trip duration:", clean_df['trip_duration_minutes'].median())
```

```
Q.53 What is the median trip duration?
Median trip duration: 11.7
```

```
print("Q.54 Do longer trips generally take longer?")

correlation = clean_df['trip_distance'].corr(clean_df['trip_duration_minutes'])

print("Correlation between trip distance and trip duration:", correlation)
```

```
Q.54 Do longer trips generally take longer?
Correlation between trip distance and trip duration: 0.0030132613977519066
```

```
print("Q.55 Are there trips with very high distance but unusually low duration?")

high_distance_trips = clean_df.nlargest(10, 'trip_distance')

print(high_distance_trips[
    ['trip_distance',
     'trip_duration_minutes',
     'tpep_pickup_datetime',
     'tpep_dropoff_datetime']])
```

```
Q.55 Are there trips with very high distance but unusually low duration?
         trip_distance  trip_duration_minutes tpep_pickup_datetime  \
3275762      276423.57                    9.0  2025-01-22 21:04:00
3188501      276099.95                    7.0  2025-01-19 14:51:00
3240337      222167.49                   26.0  2025-01-21 09:19:00
3112173      206137.99                   24.0  2025-01-16 10:47:00
3374854      202771.63                   13.0  2025-01-26 17:46:00
3081597      189687.43                   18.0  2025-01-14 19:18:00
3069039      181139.99                    6.0  2025-01-13 21:11:00
3312563      168079.57                   11.0  2025-01-24 08:33:00
3195037      167452.94                   27.0  2025-01-19 18:31:00
3138510      164959.95                   15.0  2025-01-17 14:25:00

        tpep_dropoff_datetime
3275762   2025-01-22 21:13:00
3188501   2025-01-19 14:58:00
3240337   2025-01-21 09:45:00
3112173   2025-01-16 11:11:00
3374854   2025-01-26 17:59:00
3081597   2025-01-14 19:36:00
3069039   2025-01-13 21:17:00
3312563   2025-01-24 08:44:00
3195037   2025-01-19 18:58:00
3138510   2025-01-17 14:40:00
```

```
print("Q.56 Are there trips with very low distance but unusually high duration?")

low_distance_trips = clean_df[
    clean_df['trip_distance'] <= 1].sort_values('trip_duration_minutes', ascending=False).head(10)

print(low_distance_trips[
    ['trip_distance',
     'trip_duration_minutes',
     'tpep_pickup_datetime',
     'tpep_dropoff_datetime']])
```

```
Q.56 Are there trips with very low distance but unusually high duration?
         trip_distance  trip_duration_minutes tpep_pickup_datetime  \
2460536           0.00            4184.000000  2025-01-27 11:58:55
898201            0.04            3405.466667  2025-01-11 08:04:00
2147536           0.00            3062.383333  2025-01-24 07:07:21
85681             0.00            2876.500000  2025-01-02 09:40:31
55016             0.66            2485.733333  2025-01-01 17:49:26
2745903           0.00            1746.716667  2025-01-30 11:24:49
2538084           0.00            1699.633333  2025-01-28 10:48:56
2548395           0.46            1565.316667  2025-01-28 12:59:43
1832541           0.44            1466.700000  2025-01-21 05:22:58
1832540           0.44            1466.700000  2025-01-21 05:22:58

        tpep_dropoff_datetime
2460536   2025-01-30 09:42:55
898201    2025-01-13 16:49:28
2147536   2025-01-26 10:09:44
85681     2025-01-04 09:37:01
55016     2025-01-03 11:15:10
2745903   2025-01-31 16:31:32
2538084   2025-01-29 15:08:34
2548395   2025-01-29 15:05:02
1832541   2025-01-22 05:49:40
1832540   2025-01-22 05:49:40
```

```
print("Q.57 What does this tell you about potential data anomalies?")

print("\nSome trips have unusual distance-duration combinations.")
print("These records should be investigated as potential anomalies.")
```

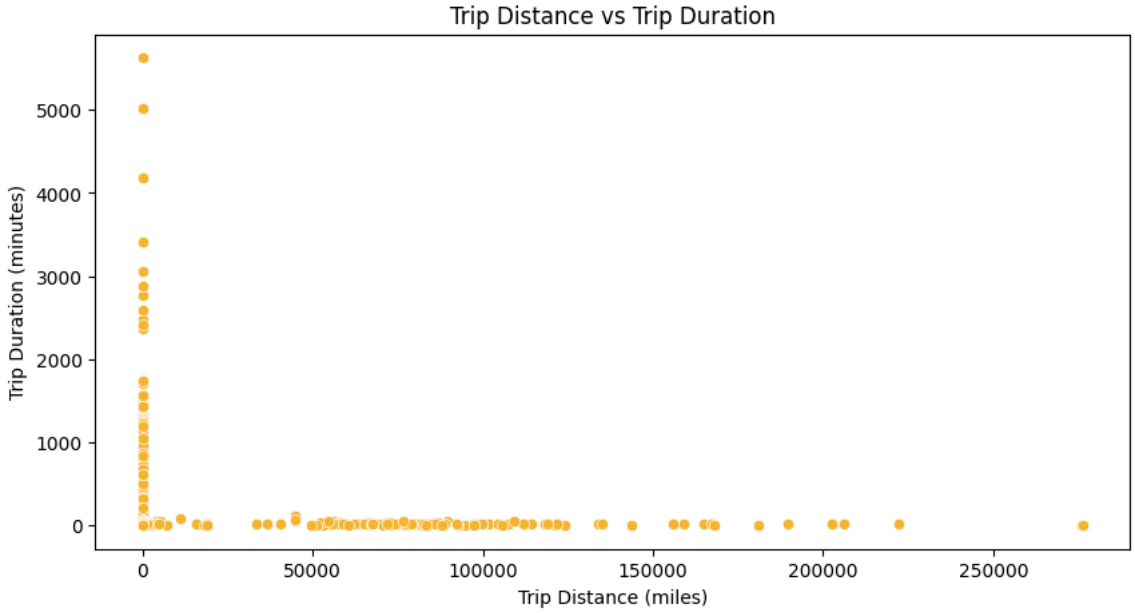
```
Q.57 What does this tell you about potential data anomalies?

Some trips have unusual distance-duration combinations.
These records should be investigated as potential anomalies.
```

```
#Viusalization part
plt.figure(figsize=(10, 5))

sns.scatterplot(x=clean_df['trip_distance'],y=clean_df['trip_duration_minutes'],color='#F7B731')
plt.title("Trip Distance vs Trip Duration")
plt.xlabel("Trip Distance (miles)")
plt.ylabel("Trip Duration (minutes)")

plt.show()
print('Insights--> \n The pattern indicates that the longer trip requied more time.\n Some extreme values
```




```
Insights-->
 The pattern indicates that the longer trip requied more time.
 Some extreme values indicate potential data anomalies.
```

Phase F — Passenger Analysis

```
print("Q.58 What is the most common passenger count?")
print(clean_df['passenger_count'].value_counts())
print('\n - 2322429 - The largest count is the most common passenger count')
```

```
Q.58 What is the most common passenger count?
passenger_count
1.0    2322429
2.0     407761
3.0      91409
4.0      59009
5.0      17786
6.0      12004
8.0         11
7.0          4
9.0          3
Name: count, dtype: int64

 - 2322429 - The largest count is the most common passenger count
```

```
print("Q.59 What percentage of trips have one passenger?")

one_passenger = (clean_df['passenger_count'] == 1).sum()
total_passenger = clean_df['passenger_count'].notna().sum()

print("Percentage:", (one_passenger / total_passenger) * 100)
```

```
Q.59 What percentage of trips have one passenger?
Percentage: 79.79714927350592
```

```
print("Q.60 How does trip distance vary with passenger count?")
print(clean_df.groupby('passenger_count')['trip_distance'].mean())

print('---'*20)
print("Passenger count 9 has the highest average distance, but only a few trips are recorded.")
print("For the main passenger groups, average trip distances are fairly similar.")
```

```
Q.60 How does trip distance vary with passenger count?
passenger_count
1.0     3.094353
2.0     3.681512
3.0     3.483201
4.0     3.723703
5.0     2.938389
6.0     3.070723
7.0     0.005000
8.0     6.340000
9.0    14.863333
Name: trip_distance, dtype: float64
------------------------------------------------------------
Passenger count 9 has the highest average distance, but only a few trips are recorded.
For the main passenger groups, average trip distances are fairly similar.
```

```
print("Q.61 How does trip duration vary with passenger count?")
print(clean_df.groupby('passenger_count')['trip_duration_minutes'].mean())
print('---'*20)
print("Trip duration is fairly similar for the main passenger-count groups.")
print("Very small passenger groups can show unusual average values.")
```

```
Q.61 How does trip duration vary with passenger count?
passenger_count
1.0    14.712964
2.0    15.833111
3.0    15.699438
4.0    16.447093
5.0    19.002863
6.0    21.826842
7.0     0.529167
8.0    12.712121
9.0    25.033333
Name: trip_duration_minutes, dtype: float64
------------------------------------------------------------
Trip duration is fairly similar for the main passenger-count groups.
Very small passenger groups can show unusual average values.
```

```
print("Q.62 Are passenger counts distributed differently during weekdays and weekends?")

passenger_weekend = (clean_df.groupby('is_weekend')['passenger_count'].value_counts(normalize=True) * 100
print(passenger_weekend)

print('\n1 passenger is the most common for both weekdays and weekends.')
```

```
Q.62 Are passenger counts distributed differently during weekdays and weekends?
is_weekend  passenger_count
False       1.0               81.892099
            2.0               12.733682
            3.0                2.691908
            4.0                1.674063
            5.0                0.593891
            6.0                0.413718
            8.0                0.000320
            7.0                0.000183
            9.0                0.000137
True        1.0               73.446884
            2.0               17.880432
            3.0                4.501308
            4.0                3.098891
            5.0                0.663326
            6.0                0.408605
            8.0                0.000554
Name: proportion, dtype: float64

1 passenger is the most common for both weekdays and weekends.
```

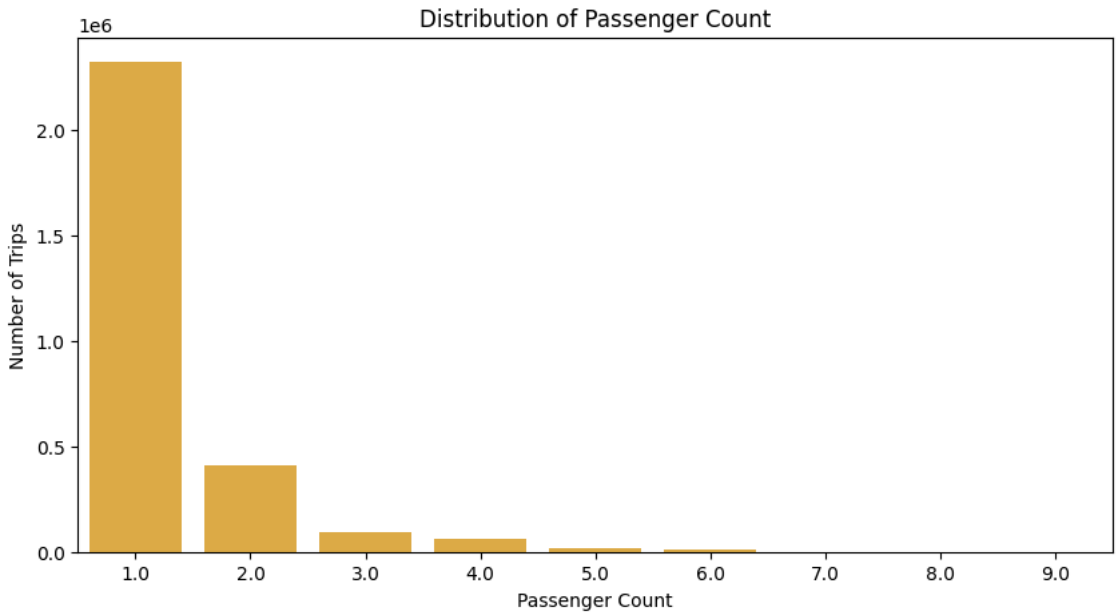
```
#Visualization

plt.figure(figsize=(10, 5))

sns.countplot(data=clean_df, x='passenger_count',color='#F7B731')

plt.title("Distribution of Passenger Count")
plt.xlabel("Passenger Count")
plt.ylabel("Number of Trips")

plt.show()
print('Insights-->\n The lower number of passenger are having most number of trip and other are having cc
```



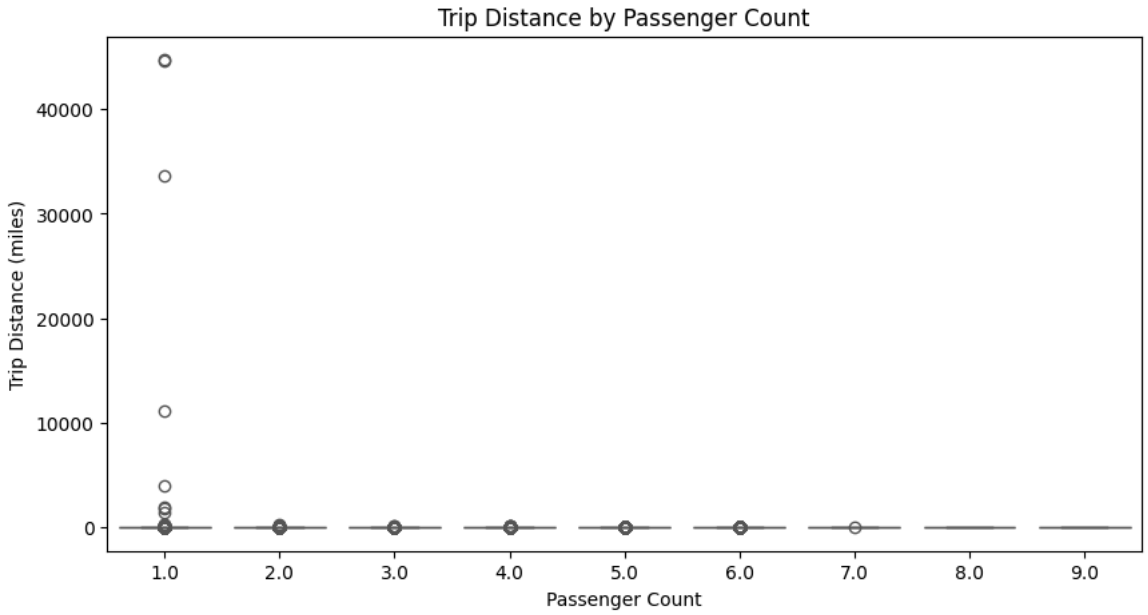

```
Insights-->
 The lower number of passenger are having most number of trip and other are having comaprtively low
```

```
plt.figure(figsize=(10, 5))

sns.boxplot(data=clean_df, x='passenger_count', y='trip_distance',color='#F7B731')

plt.title("Trip Distance by Passenger Count")
plt.xlabel("Passenger Count")
plt.ylabel("Trip Distance (miles)")

plt.show()
print("Insight-->\n Trip distance varies across passenger counts, with several extreme outliers.\n Most t
```




```
Insight-->
 Trip distance varies across passenger counts, with several extreme outliers.
 Most trips are concentrated at relatively low distances.
```

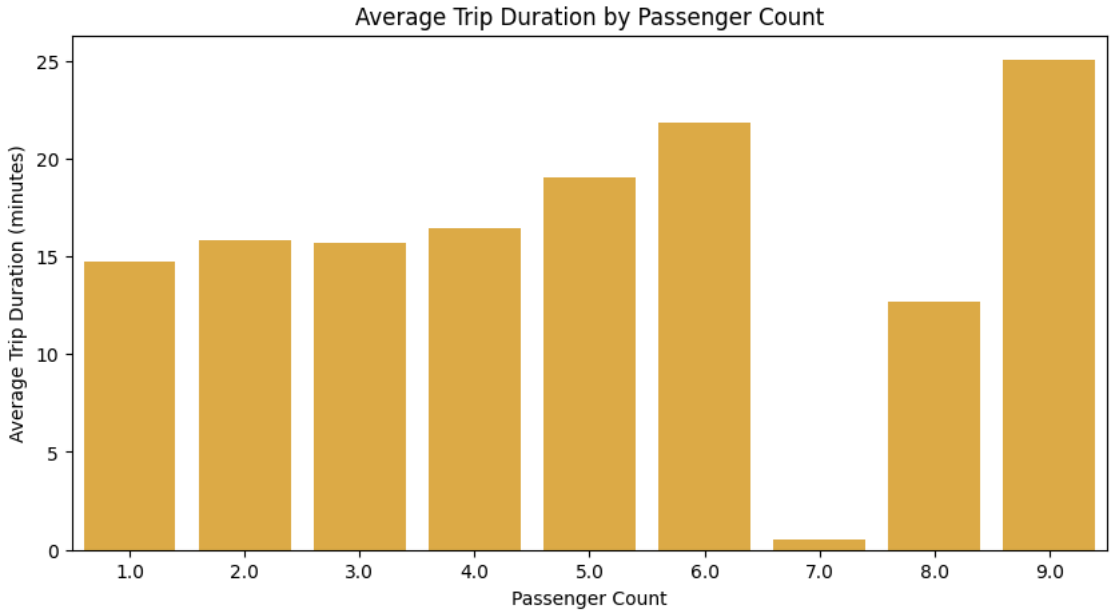
```
avg_duration_passenger = clean_df.groupby('passenger_count')['trip_duration_minutes'].mean()

plt.figure(figsize=(10, 5))

sns.barplot(x=avg_duration_passenger.index,y=avg_duration_passenger.values,color='#F7B731')

plt.title("Average Trip Duration by Passenger Count")
plt.xlabel("Passenger Count")
plt.ylabel("Average Trip Duration (minutes)")

plt.show()
print('Insights-->\n The 9th passenger count  has the higest avg trip duration followed by 6.0 and 7.0 ha
```




```
Insights-->
 The 9th passenger count  has the higest avg trip duration followed by 6.0 and 7.0 has lowest of all at slighlty above
```

Phase G — Pickup & Drop-off Location Analysis

```
print("Q.63 What are the top 10 pickup locations?")
print(clean_df['PULocationID'].value_counts().head(10))
print('\n The above location ID are the TOP 10')
```

```
Q.63 What are the top 10 pickup locations?
PULocationID
```

```
161    169977
237    163702
236    155644
132    146137
230    125828
186    119131
162    117929
142    110584
239     96614
163     95906
Name: count, dtype: int64

 The above location ID are the TOP 10
```

```
print("Q.64 Which pickup location generates the highest number of trips?")
pickup_trips = clean_df['PULocationID'].value_counts()

print(f'\n {pickup_trips.idxmax()} receives the highest number of trips as {pickup_trips.max()}')
```

```
Q.64 Which pickup location generates the highest number of trips?

 161 receives the highest number of trips as 169977
```

```
print("Top 10 Pickup Locations")

top_pickup = clean_df['PULocationID'].value_counts().head(10)

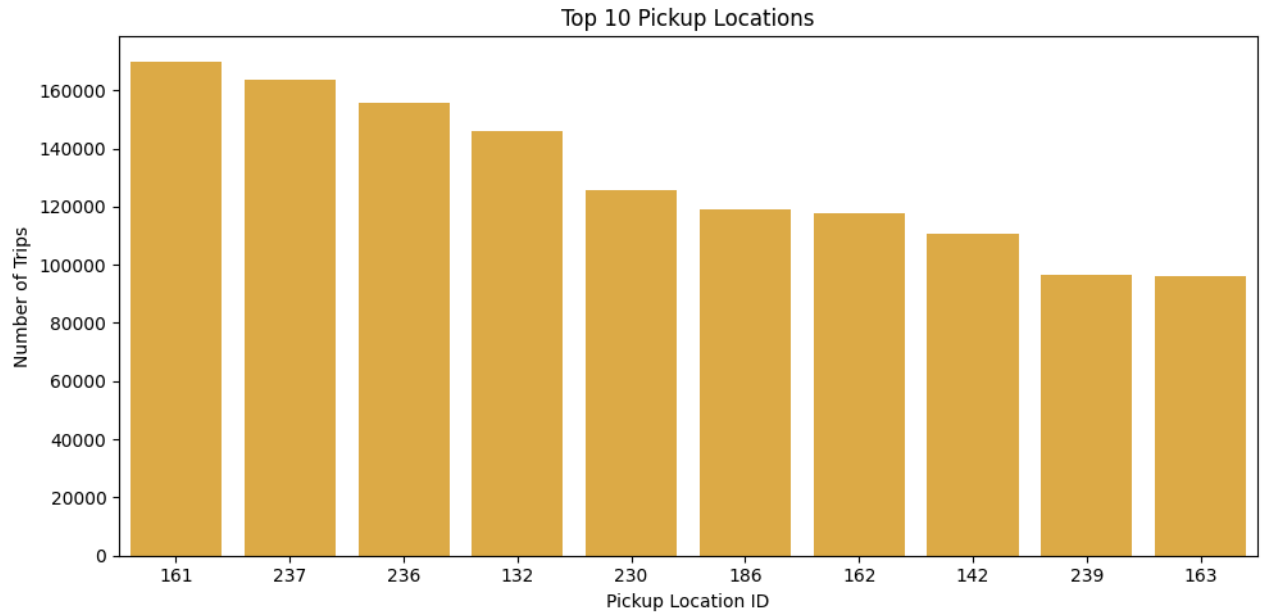
plt.figure(figsize=(10, 5))

sns.barplot(
    x=top_pickup.index.astype(str),
    y=top_pickup.values,
    color='#F7B731'
)

plt.title('Top 10 Pickup Locations')
plt.xlabel('Pickup Location ID')
plt.ylabel('Number of Trips')

plt.tight_layout()
plt.show()
print('---'*20)
print("Insight:")
print("A small group of pickup locations accounts for a large share of taxi trips.")
```

```
Top 10 Pickup Locations
```




```
------------------------------------------------------------
Insight:
A small group of pickup locations accounts for a large share of taxi trips.
```

```
print("Q.65 What are the top 10 drop-off locations?")
print(clean_df['DOLocationID'].value_counts().head(10))
```

```
print('\n The above location ID are the TOP 10')
```

```
Q.65 What are the top 10 drop-off locations?
DOLocationID
236    161376
237    149970
161    131258
230    108177
170    100057
142     98981
239     97559
162     93798
141     92675
68      89232
Name: count, dtype: int64

 The above location ID are the TOP 10
```

```
print("Q.66 Which drop-off location receives the highest number of trips?")
dropoff_trips = clean_df['DOLocationID'].value_counts()
print(f'\n {dropoff_trips.idxmax()} receives the highest number of trips as {dropoff_trips.max()}')
```

```
Q.66 Which drop-off location receives the highest number of trips?

 236 receives the highest number of trips as 161376
```

```
print("Top 10 Drop-off Locations")

top_dropoff = clean_df['DOLocationID'].value_counts().head(10)

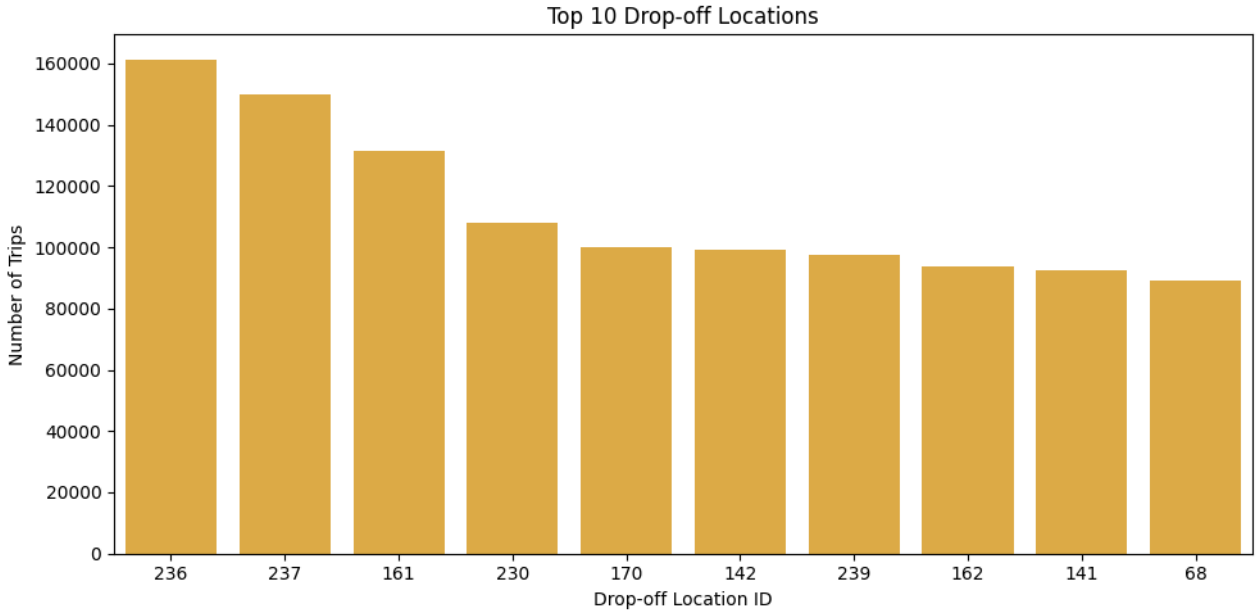
plt.figure(figsize=(10, 5))

sns.barplot(
    x=top_dropoff.index.astype(str),
    y=top_dropoff.values,
    color='#F7B731'
)

plt.title('Top 10 Drop-off Locations')
plt.xlabel('Drop-off Location ID')
plt.ylabel('Number of Trips')

plt.tight_layout()
plt.show()
print('---'*20)
print("Insight:")
print("A small group of drop-off locations receives a large share of taxi trips.")
```

```
Top 10 Drop-off Locations
```




```
------------------------------------------------------------
Insight:
A small group of drop-off locations receives a large share of taxi trips.
```

```
print("Q.67 What are the most frequently travelled pickup → drop-off routes?")

route_trips = (clean_df.groupby(['PULocationID', 'DOLocationID']).size().sort_values(ascending=False).hea

print(route_trips)
```

```
Q.67 What are the most frequently travelled pickup → drop-off routes?
   PULocationID  DOLocationID  Number of Trips
0           237           236            24839
1           236           237            21843
2           236           236            18221
3           237           237            17441
4           161           237            11463
5           237           161            10168
6           161           236             9845
7           239           238             9595
8           142           239             9141
9           239           142             8844
```

```
print("Q.68 Are the most common pickup locations also the most common drop-off locations?")

print("Top Pickup Locations:")
print(clean_df['PULocationID'].value_counts().head(5))

print("\nTop Drop-off Locations:")
print(clean_df['DOLocationID'].value_counts().head(5))
```

```
Q.68 Are the most common pickup locations also the most common drop-off locations?
Top Pickup Locations:
PULocationID
161    169977
237    163702
236    155644
132    146137
230    125828
Name: count, dtype: int64

Top Drop-off Locations:
DOLocationID
236    161376
237    149970
161    131258
230    108177
170    100057
Name: count, dtype: int64
```

```
print("Q.69 Which routes have the longest average trip distance?")
print('\n The routes listed below have the longest avg trip distance')
avg_distance_route = clean_df.groupby('route')['trip_distance'].mean()

print(avg_distance_route.sort_values(ascending=False).head(10))
```

```
Q.69 Which routes have the longest average trip distance?

 The routes listed below have the longest avg trip distance
route
63 → 47      76492.140000
131 → 50     42978.440000
94 → 168     39528.840000
146 → 45     39014.475000
133 → 13     28595.850000
247 → 239    27914.090000
52 → 144     27773.732500
242 → 78     17366.321429
212 → 169    13059.138750
164 → 83     12809.902857
Name: trip_distance, dtype: float64
```

```
print("Q.70 Which routes have the longest average trip duration?")
print('\n The routes listed below have the longest avg trip duration')
avg_duration_route = clean_df.groupby('route')['trip_duration_minutes'].mean()

print(avg_duration_route.sort_values(ascending=False).head(10))
```

```
Q.70 Which routes have the longest average trip duration?

 The routes listed below have the longest avg trip duration
route
265 → 141    2764.533333
218 → 193    2528.091667
148 → 91     2419.083333
207 → 216    1746.716667
207 → 146    1699.633333
```

```
207 → 142    1541.783333
193 → 181     989.077778
108 → 13      897.983333
207 → 226     889.850000
231 → 193     878.427778
Name: trip_duration_minutes, dtype: float64
```

Phase H — Payment Type Analysis

```
print("Q.71 What are the different payment types present?")
payment_trips = clean_df['payment_type'].value_counts()
payment_trips_named = payment_trips.rename(index=payment_names)

print(payment_trips_named)
```

```
Q.71 What are the different payment types present?
payment_type
Credit Card    2444387
Flex Fare       540031
Cash            390429
Dispute          76481
No Charge        23773
Unknown              1
Name: count, dtype: int64
```

```
print("Q.72 Which payment type occurs most frequently?")

payment_trips = clean_df['payment_type'].value_counts()

print("Most common payment type:", payment_names[payment_trips.idxmax()])
print("Number of trips:", payment_trips.max())
print('---'*20)
print(f'\nThe most frequently used payment type is { payment_names[payment_trips.idxmax()]}, with number
```

```
Q.72 Which payment type occurs most frequently?
Most common payment type: Credit Card
Number of trips: 2444387
------------------------------------------------------------

The most frequently used payment type is Credit Card, with number of trip as 2444387
```

```
print("Q.73 What percentage of trips belong to each payment type?")

payment_percentage = clean_df['payment_type'].value_counts(normalize=True) * 100
print(payment_percentage.rename(index=payment_names))
```

```
Q.73 What percentage of trips belong to each payment type?
payment_type
Credit Card    70.340007
Flex Fare      15.540004
Cash           11.235037
Dispute         2.200827
No Charge       0.684095
Unknown         0.000029
Name: proportion, dtype: float64
```

```
print("Q.74 Does trip distance vary across payment types?")

avg_distance_payment = clean_df.groupby('payment_type')['trip_distance'].mean()
print(avg_distance_payment.rename(index=payment_names))
print('---'*20)
print('\nYes, Fare Flex has higest trip distance comapred to others')
```

```
Q.74 Does trip distance vary across payment types?
payment_type
Flex Fare      20.317599
Credit Card     3.195515
Cash            3.140658
No Charge       2.546687
Dispute         3.623720
Unknown         0.000000
Name: trip_distance, dtype: float64
------------------------------------------------------------

Yes, Fare Flex has higest trip distance comapred to others
```

```
print("Q.75 Does trip duration vary across payment types?")

avg_duration_payment = clean_df.groupby('payment_type')['trip_duration_minutes'].mean()
print(avg_duration_payment.rename(index=payment_names))

print('---'*20)
print("Trip durations are fairly similar for the main payment types.")
print("Some smaller payment groups show different values.")
```

```
Q.75 Does trip duration vary across payment types?
payment_type
Flex Fare      15.361382
Credit Card    15.027335
Cash           15.099025
No Charge      11.197425
Dispute        13.775009
Unknown         0.316667
Name: trip_duration_minutes, dtype: float64
------------------------------------------------------------
Trip durations are fairly similar for the main payment types.
Some smaller payment groups show different values.
```

```
print("Q.76 Does passenger count vary across payment types?")

avg_passenger_payment = clean_df.groupby('payment_type')['passenger_count'].mean()
print(avg_passenger_payment.rename(index=payment_names))

print('---'*20)
print("Passenger count varies slightly across the main payment types.")
print("Flex Fare has missing passenger-count values, so it cannot be compared directly.")
```

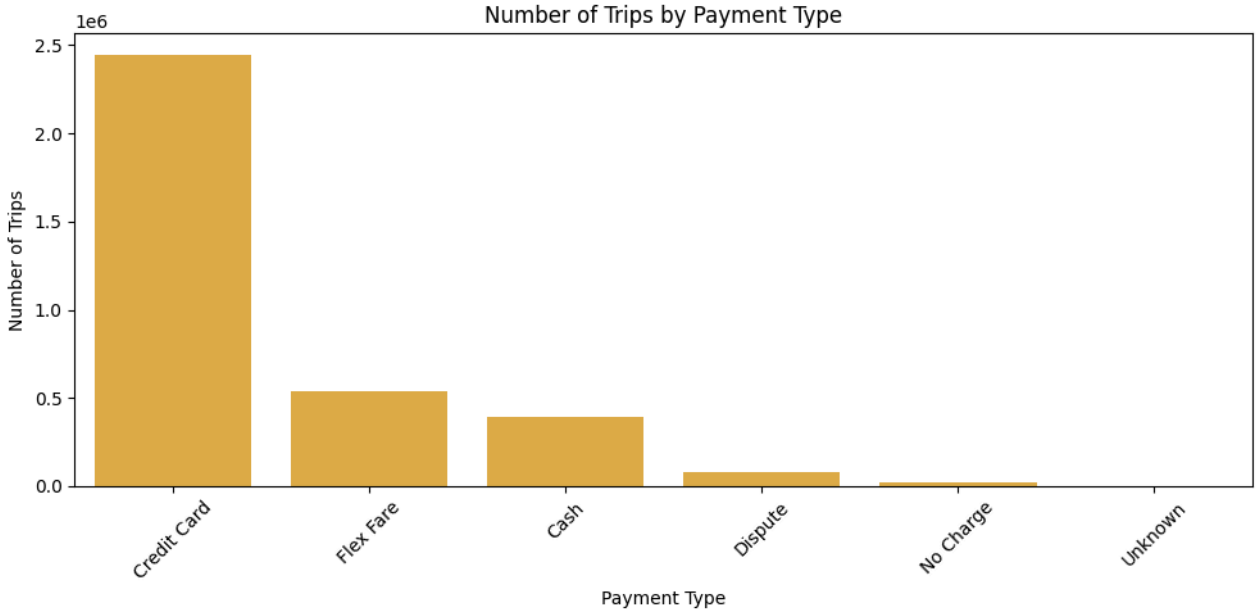
```
Q.76 Does passenger count vary across payment types?
payment_type
Flex Fare           NaN
Credit Card    1.296435
Cash           1.378084
No Charge      1.317729
Dispute        1.350250
Unknown        1.000000
Name: passenger_count, dtype: float64
------------------------------------------------------------
Passenger count varies slightly across the main payment types.
Flex Fare has missing passenger-count values, so it cannot be compared directly.
```

```
#Visualization
# Payment Type Count Plot
payment_counts = clean_df['payment_type'].value_counts().rename(index=payment_names)

plt.figure(figsize=(10, 5))
sns.barplot(x=payment_counts.index,y=payment_counts.values,color='#F7B731')

plt.title('Number of Trips by Payment Type')
plt.xlabel('Payment Type')
plt.ylabel('Number of Trips')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

print("Insight-->\n Credit Card is the most frequently used payment type.\n Other payment types account f
```

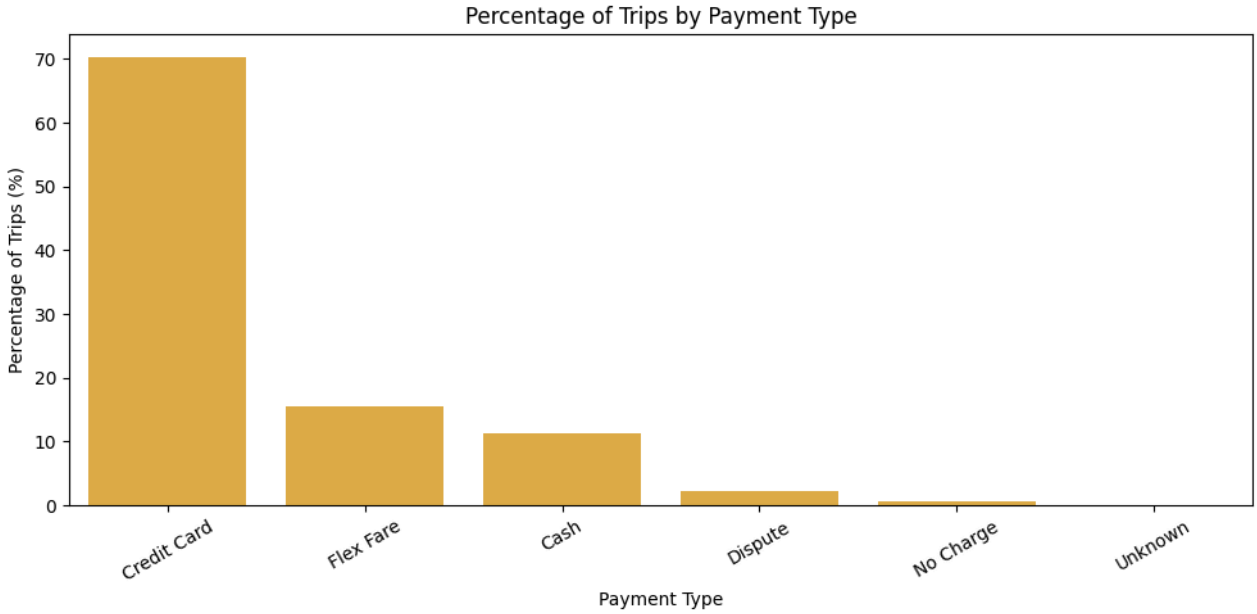



```
Insight-->
 Credit Card is the most frequently used payment type.
 Other payment types account for a much smaller share of the trips.
```

```
#2. Payment Type Percentage Analysis
payment_percentage = clean_df['payment_type'].value_counts(normalize=True) * 100
payment_percentage = payment_percentage.rename(index=payment_names)

plt.figure(figsize=(10, 5))
sns.barplot(x=payment_percentage.index,y=payment_percentage.values,color='#F7B731')

plt.title('Percentage of Trips by Payment Type')
plt.xlabel('Payment Type')
plt.ylabel('Percentage of Trips (%)')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
print("Insight-->\n Credit Card accounts for the largest percentage of trips.\n The remaining payment typ
```



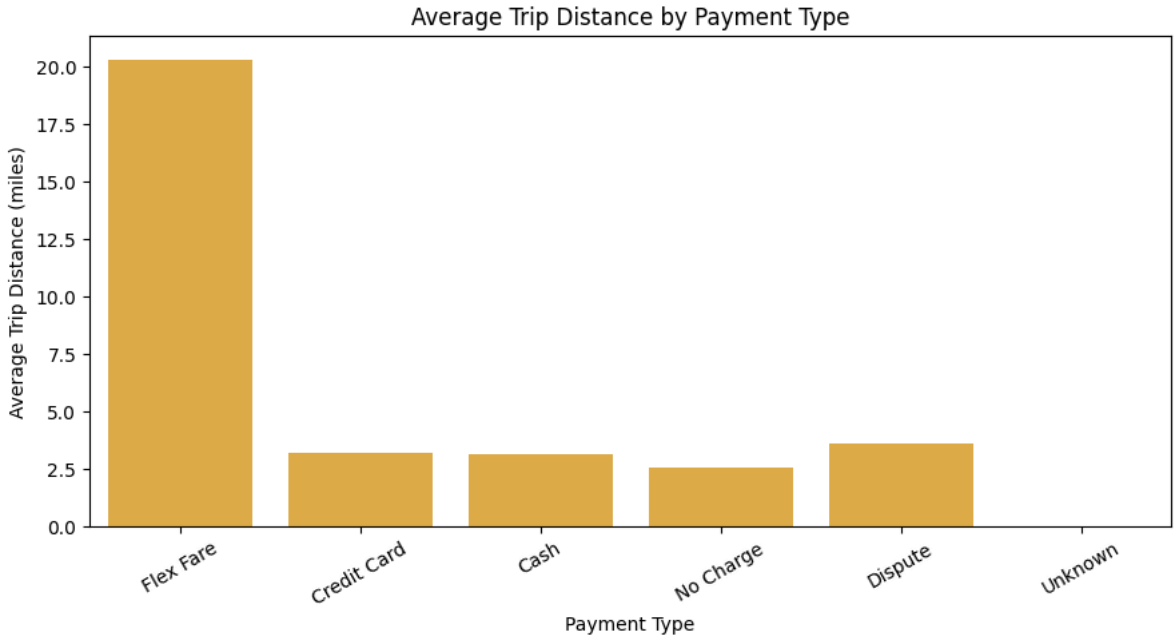

```
Insight-->
 Credit Card accounts for the largest percentage of trips.
 The remaining payment types represent much smaller shares.
```

```
#3. Payment Type vs Trip Distance
avg_distance_payment = clean_df.groupby('payment_type')['trip_distance'].mean()
avg_distance_payment = avg_distance_payment.rename(index=payment_names)

plt.figure(figsize=(9, 5))
```

```
sns.barplot(x=avg_distance_payment.index, y=avg_distance_payment.values,color='#F7B731')

plt.title('Average Trip Distance by Payment Type')
plt.xlabel('Payment Type')
plt.ylabel('Average Trip Distance (miles)')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
print("\nInsight-->\n Average trip distance varies across payment types.\n Some payment types show higher
```



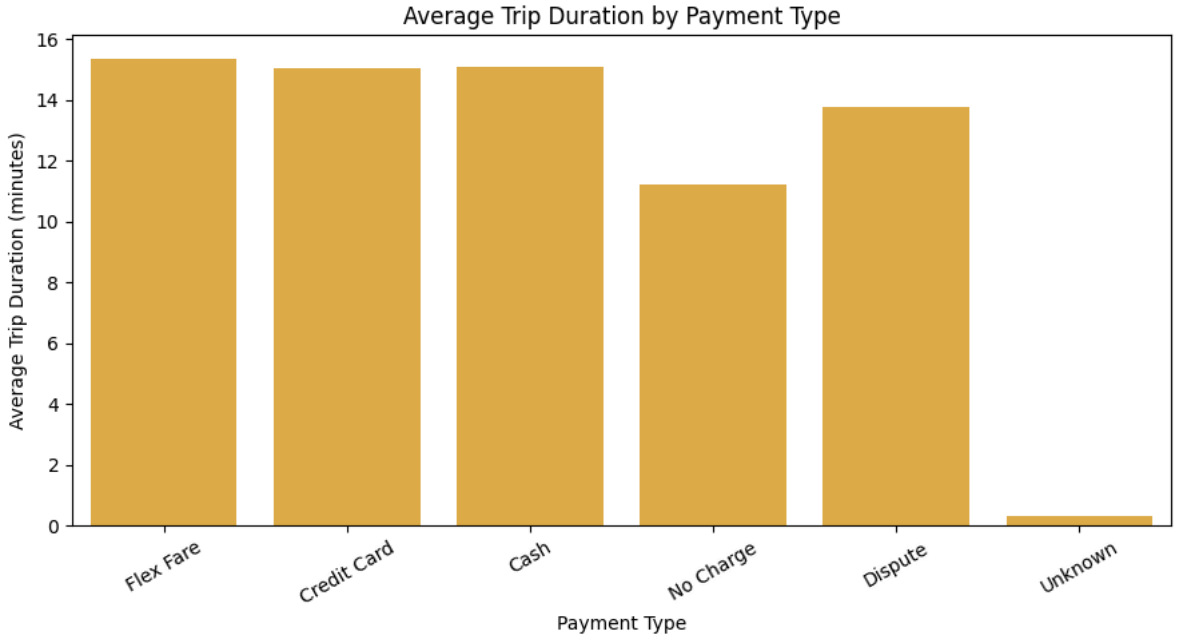

```
Insight-->
 Average trip distance varies across payment types.
 Some payment types show higher average distances than others.
```

```
#4. Payment Type vs Trip Duration
avg_duration_payment = clean_df.groupby('payment_type')['trip_duration_minutes'].mean()
avg_duration_payment = avg_duration_payment.rename(index=payment_names)

plt.figure(figsize=(9, 5))
sns.barplot(x=avg_duration_payment.index,y=avg_duration_payment.values,color='#F7B731')

plt.title('Average Trip Duration by Payment Type')
plt.xlabel('Payment Type')
plt.ylabel('Average Trip Duration (minutes)')
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

print("Insight-->\n Average trip duration differs across payment types.")
```




```
Insight-->
 Average trip duration differs across payment types.
```

Phase I — Vendor Analysis

```
print("Q.77 How many trips are recorded by each vendor?")

vendor_trips = clean_df['VendorID'].value_counts()
print(vendor_trips)
```

```
Q.77 How many trips are recorded by each vendor?
VendorID
2    2719860
1     753665
7       1206
6        371
Name: count, dtype: int64
```

```
print("Q.78 What percentage of trips does each vendor represent?")

vendor_percentage = clean_df['VendorID'].value_counts(normalize=True) * 100
print(vendor_percentage)
```

```
Q.78 What percentage of trips does each vendor represent?
VendorID
2    78.267055
1    21.687565
7     0.034704
6     0.010676
Name: proportion, dtype: float64
```

```
print("Q.79 Do vendors have different average trip distances?")

avg_distance_vendor = clean_df.groupby('VendorID')['trip_distance'].mean()
print(avg_distance_vendor)
```

```
Q.79 Do vendors have different average trip distances?
VendorID
1    3.198076
2    6.592589
6    9.537089
7    1.962902
Name: trip_distance, dtype: float64
```

```
print("Q.80 Do vendors have different average trip durations?")

avg_duration_vendor = clean_df.groupby('VendorID')['trip_duration_minutes'].mean()
print(avg_duration_vendor)

print('---'*20)
```

```
print("Trip duration varies across vendors.")
print("Vendor 6 has a much higher average, but it represents very few trips.")
```

```
Q.80 Do vendors have different average trip durations?
VendorID
1    15.698637
2    14.852311
6    41.360737
7     0.000000
Name: trip_duration_minutes, dtype: float64
------------------------------------------------------------
Trip duration varies across vendors.
Vendor 6 has a much higher average, but it represents very few trips.
```

```
print("Q.81 Do vendors show different passenger-count distributions?")
vendor_passenger_percentage = (
    clean_df.groupby('VendorID')['passenger_count']
    .value_counts(normalize=True) * 100
)

print(vendor_passenger_percentage)
print("\nPassenger-count distributions are broadly similar across vendors.\nOne-passenger trips are the m
```

```
Q.81 Do vendors show different passenger-count distributions?
VendorID  passenger_count
1         1.0               86.312634
          2.0               10.297980
          3.0                2.163421
          4.0                1.126727
          5.0                0.070528
          6.0                0.028710
2         1.0               77.956132
          2.0               15.058279
          3.0                3.417012
          4.0                2.282563
          5.0                0.764131
          6.0                0.521089
          8.0                0.000485
          7.0                0.000176
          9.0                0.000132
7         1.0               80.099502
          2.0               15.920398
          3.0                2.902156
          4.0                0.995025
          5.0                0.082919
Name: proportion, dtype: float64

Passenger-count distributions are broadly similar across vendors.
One-passenger trips are the most common for all vendors, while higher passenger counts occur less frequently.
```

```
print("Q.82 Are there differences in payment-type distribution between vendors?")

vendor_payment = (
    clean_df.groupby('VendorID')['payment_type']
    .value_counts(normalize=True) * 100
)

print(vendor_payment)
print("\nPayment-type distributions show differences across vendors.\nCredit Card is the dominant payment
```

```
Q.82 Are there differences in payment-type distribution between vendors?
VendorID  payment_type
1         1               75.040767
          0               11.703343
          2               11.453497
          3                1.115748
          4                0.686512
          5                0.000133
2         1               69.039583
          0               16.598501
          2               11.176311
          4                2.621716
          3                0.563889
6         0              100.000000
7         1               87.147595
          2               10.613599
          3                2.238806
Name: proportion, dtype: float64

Payment-type distributions show differences across vendors.
Credit Card is the dominant payment type, while the remaining payment types represent smaller shares.
```

Phase J — Rate Code Analysis

```
print("Q.83 Which rate codes are most frequently used?")

ratecode_trips = clean_df['RatecodeID'].value_counts()
print(ratecode_trips)
print('\n1 Rating is given more frequently.')
```

```
Q.83 Which rate codes are most frequently used?
RatecodeID
1.0     2756471
2.0       94420
99.0      41958
5.0       26501
3.0        8622
4.0        7092
6.0           7
Name: count, dtype: int64

1 Rating is given more frequently.
```

```
print("Q.84 What percentage of trips belongs to each rate code?")

ratecode_percentage = clean_df['RatecodeID'].value_counts(normalize=True) * 100
print(ratecode_percentage)
```

```
Q.84 What percentage of trips belongs to each rate code?
RatecodeID
1.0     93.914968
2.0      3.216958
99.0     1.429540
5.0      0.902908
3.0      0.293758
4.0      0.241630
6.0      0.000238
Name: proportion, dtype: float64
```

```
print("Q.85 How does trip distance vary across rate codes?")

avg_distance_ratecode = clean_df.groupby('RatecodeID')['trip_distance'].mean()
print(avg_distance_ratecode)

print('\nRate code 4 has the highest average trip distance.')
```

```
Q.85 How does trip distance vary across rate codes?
RatecodeID
1.0      2.495872
2.0     17.180332
3.0     14.347970
4.0     20.961847
5.0      5.558306
6.0      0.457143
99.0    10.804326
Name: trip_distance, dtype: float64

Rate code 4 has the highest average trip distance.
```

```
print("Q.86 How does trip duration vary across rate codes?")

avg_duration_ratecode = clean_df.groupby('RatecodeID')['trip_duration_minutes'].mean()
print(avg_duration_ratecode)

print('\nSome rate codes have higher average trip durations than others.')
```

```
Q.86 How does trip duration vary across rate codes?
RatecodeID
1.0     13.404256
2.0     43.934370
3.0     30.248846
4.0     40.308230
5.0     15.236899
6.0      0.733333
99.0    45.289257
Name: trip_duration_minutes, dtype: float64

Some rate codes have higher average trip durations than others.
```

```
print("Q.87 Are some rate codes associated with unusually long trips?")
```

```
avg_distance_ratecode = clean_df.groupby('RatecodeID')['trip_distance'].mean()
print(avg_distance_ratecode.sort_values(ascending=False))

print('\nRate codes 4, 2 and 3 have higher average trip distances.')
```

```
Q.87 Are some rate codes associated with unusually long trips?
RatecodeID
4.0     20.961847
2.0     17.180332
3.0     14.347970
99.0    10.804326
5.0      5.558306
1.0      2.495872
6.0      0.457143
Name: trip_distance, dtype: float64

Rate codes 4, 2 and 3 have higher average trip distances.
```

```
print("Q.88 Are there rare rate codes that require further investigation?")

ratecode_trips = clean_df['RatecodeID'].value_counts()
print(ratecode_trips.sort_values())

print('\nRare rate codes have very few trips and may require further investigation.')
```

```
Q.88 Are there rare rate codes that require further investigation?
RatecodeID
6.0           7
4.0        7092
3.0        8622
5.0       26501
99.0      41958
2.0       94420
1.0     2756471
Name: count, dtype: int64

Rare rate codes have very few trips and may require further investigation.
```

Phase J Insight:

- Rate code 1 is the most frequently used rate code.
- Some rate codes occur much less frequently and may require further investigation.

Phase K — Store & Forward Analysis

```
print("Q.89 What percentage of trips are marked as store-and-forward?")

store_forward_percentage = clean_df['store_and_fwd_flag'].value_counts(normalize=True) * 100
print(store_forward_percentage)

print('\nMost trips are not marked as store-and-forward.')
print("Missing values are excluded from this percentage.")
```

```
Q.89 What percentage of trips are marked as store-and-forward?
store_and_fwd_flag
N    99.739495
Y     0.260505
Name: proportion, dtype: float64

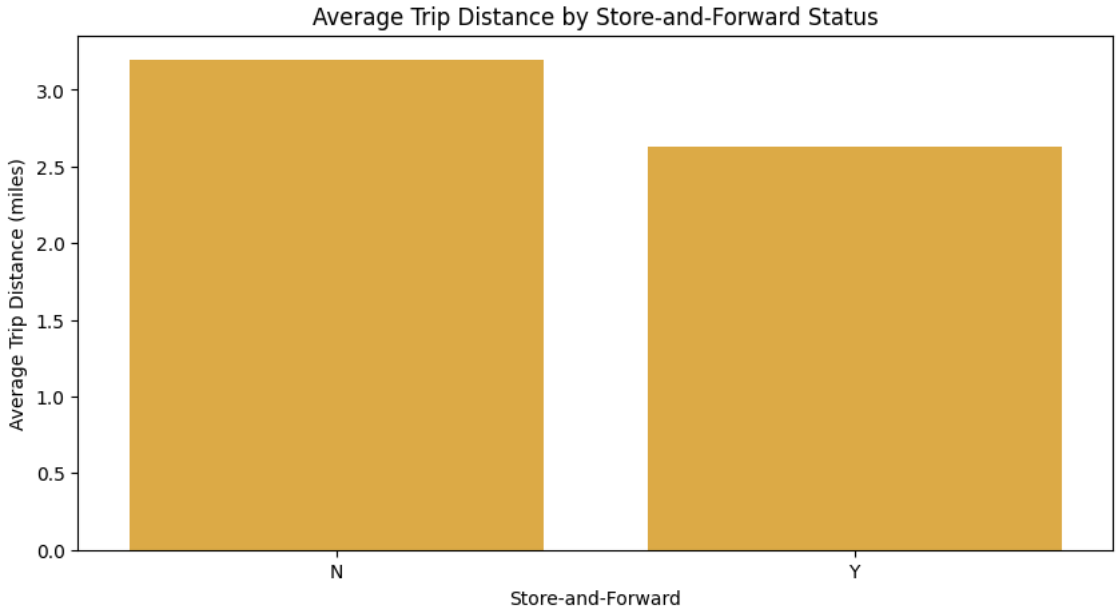
Most trips are not marked as store-and-forward.
Missing values are excluded from this percentage.
```

```
print("Q.90 How does trip distance vary between the two groups?")

avg_distance_store = clean_df.groupby('store_and_fwd_flag')['trip_distance'].mean()

plt.figure(figsize=(10, 5))
sns.barplot(x=avg_distance_store.index, y=avg_distance_store.values,color='#F7B731')
plt.title('Average Trip Distance by Store-and-Forward Status')
plt.xlabel('Store-and-Forward')
plt.ylabel('Average Trip Distance (miles)')
plt.show()
print('\nTrip distance varies between the two store-and-forward groups.')
```

```
Q.90 How does trip distance vary between the two groups?
```



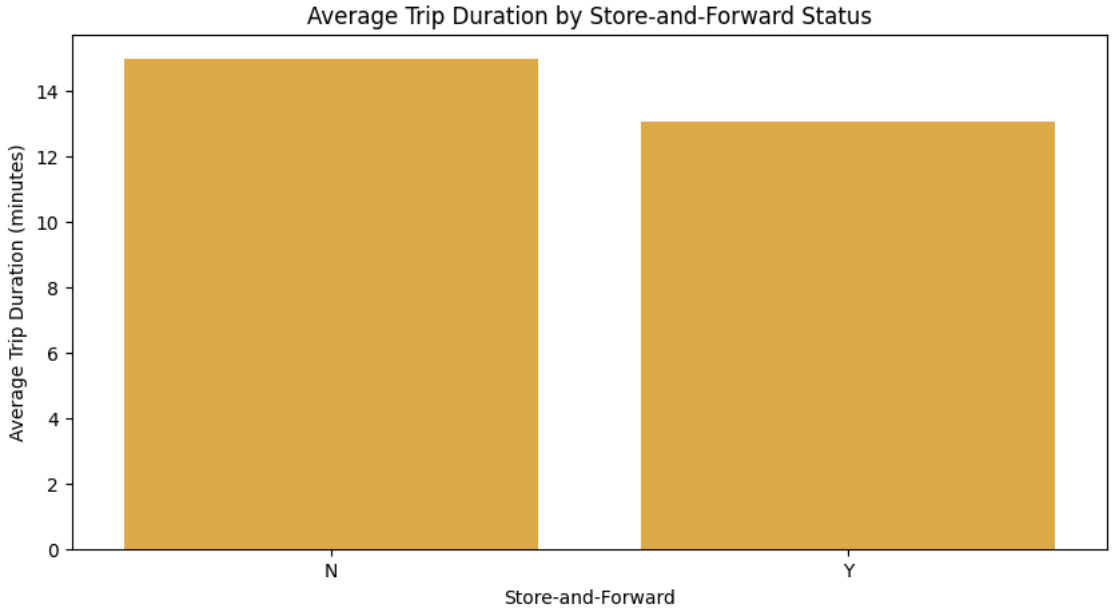

```
Trip distance varies between the two store-and-forward groups.
```

```
print("Q.91 How does trip duration vary between the two groups?")

avg_duration_store = clean_df.groupby('store_and_fwd_flag')['trip_duration_minutes'].mean()
print(avg_duration_store)

plt.figure(figsize=(10, 5))
sns.barplot(x=avg_duration_store.index, y=avg_duration_store.values,color='#F7B731')
plt.title('Average Trip Duration by Store-and-Forward Status')
plt.xlabel('Store-and-Forward')
plt.ylabel('Average Trip Duration (minutes)')
plt.show()
print('\nTrip duration varies between the two store-and-forward groups.')
```

```
Q.91 How does trip duration vary between the two groups?
store_and_fwd_flag
N    14.978195
Y    13.065540
Name: trip_duration_minutes, dtype: float64
```




```
Trip duration varies between the two store-and-forward groups.
```

```
print("Q.92 Are there meaningful differences in the characteristics of these trips?")
print("\nThere are some differences in trip distance, duration and passenger count.")
print("Overall, the two groups show some variation in trip characteristics.")
```

```
Q.92 Are there meaningful differences in the characteristics of these trips?

There are some differences in trip distance, duration and passenger count.
Overall, the two groups show some variation in trip characteristics.
```

Phase L — NumPy Statistical Analysis

```
print("Trip Distance Statistics")

distance = clean_df['trip_distance'].dropna()

print("Mean:", np.mean(distance))
print("Median:", np.median(distance))
print("Minimum:", np.min(distance))
print("Maximum:", np.max(distance))
print("Variance:", np.var(distance))
print("Standard Deviation:", np.std(distance))

print("\nPercentiles:")
print("25th:", np.percentile(distance, 25))
print("50th:", np.percentile(distance, 50))
print("75th:", np.percentile(distance, 75))
print("90th:", np.percentile(distance, 90))
print("95th:", np.percentile(distance, 95))
print("99th:", np.percentile(distance, 99))


print("\nInsight:")
print("Most trips are relatively short, with a median distance of 1.67 miles.")
print("The very high maximum and large standard deviation show the presence of extreme distance values.")
```

```
Trip Distance Statistics
Mean: 5.855109182406734
Median: 1.67
Minimum: 0.0
Maximum: 276423.57
Variance: 318786.2483033103
Standard Deviation: 564.6115906561876

Percentiles:
25th: 0.98
50th: 1.67
75th: 3.1
90th: 7.7
95th: 11.83
99th: 19.5

Insight:
Most trips are relatively short, with a median distance of 1.67 miles.
The very high maximum and large standard deviation show the presence of extreme distance values.
```

```
print("Trip Duration Statistics")

duration = clean_df['trip_duration_minutes'].dropna()

print("Mean:", np.mean(duration))
print("Median:", np.median(duration))
print("Minimum:", np.min(duration))
print("Maximum:", np.max(duration))
print("Variance:", np.var(duration))
print("Standard Deviation:", np.std(duration))

print("\nPercentiles:")
print("25th:", np.percentile(duration, 25))
print("50th:", np.percentile(duration, 50))
print("75th:", np.percentile(duration, 75))
print("90th:", np.percentile(duration, 90))
print("95th:", np.percentile(duration, 95))
print("99th:", np.percentile(duration, 99))

print("\nInsight:")
print("Most trips have a relatively short duration, with a median of 11.7 minutes.")
print("The very high maximum duration shows some extreme values in the data.")
```

```
Trip Duration Statistics
Mean: 15.033534301055512
Median: 11.7
Minimum: 0.0
Maximum: 5626.316666666667
Variance: 735.941786289345
Standard Deviation: 27.128247018363442

Percentiles:
25th: 7.283333333333333
50th: 11.7
75th: 18.333333333333332
90th: 27.55
95th: 36.2
99th: 58.916666666666664

Insight:
Most trips have a relatively short duration, with a median of 11.7 minutes.
The very high maximum duration shows some extreme values in the data.
```

```
print("Passenger Count Statistics")

passenger = clean_df['passenger_count'].dropna()

print("Mean:", np.mean(passenger))
print("Median:", np.median(passenger))
print("Minimum:", np.min(passenger))
print("Maximum:", np.max(passenger))
print("Variance:", np.var(passenger))
print("Standard Deviation:", np.std(passenger))

print("\nPercentiles:")
print("25th:", np.percentile(passenger, 25))
print("50th:", np.percentile(passenger, 50))
print("75th:", np.percentile(passenger, 75))
print("90th:", np.percentile(passenger, 90))
print("95th:", np.percentile(passenger, 95))
print("99th:", np.percentile(passenger, 99))

print("\nInsight:")
print("Most trips have 1 passenger, with a median passenger count of 1.")
print("Higher passenger counts are less common in the dataset.")
```

```
Passenger Count Statistics
Mean: 1.3088544730375313
Median: 1.0
Minimum: 1.0
Maximum: 9.0
Variance: 0.554010580798699
Standard Deviation: 0.7443188703766008

Percentiles:
25th: 1.0
50th: 1.0
75th: 1.0
90th: 2.0
95th: 3.0
99th: 5.0

Insight:
Most trips have 1 passenger, with a median passenger count of 1.
Higher passenger counts are less common in the dataset.
```

```
print("Average Speed Statistics")

speed = clean_df['average_speed_mph'].dropna()

print("Mean:", np.mean(speed))
print("Median:", np.median(speed))
print("Minimum:", np.min(speed))
print("Maximum:", np.max(speed))
print("Variance:", np.var(speed))
print("Standard Deviation:", np.std(speed))

print("\nPercentiles:")
print("25th:", np.percentile(speed, 25))
print("50th:", np.percentile(speed, 50))
print("75th:", np.percentile(speed, 75))
print("90th:", np.percentile(speed, 90))
print("95th:", np.percentile(speed, 95))
```

```
print("99th:", np.percentile(speed, 99))

print("\nInsight:")
print("Most trips have a relatively low average speed, with a median of about 9.52 mph.")
print("The very high maximum speed shows some extreme values in the data.")
```

```
Average Speed Statistics
Mean: 25.73552721478154
Median: 9.517747858017136
Minimum: 0.0
Maximum: 3722496.9
Variance: 15007009.497844677
Standard Deviation: 3873.8881627951882

Percentiles:
25th: 7.2090112640801
50th: 9.517747858017136
75th: 12.93291536050157
90th: 19.436619718309856
95th: 25.06782459959569
99th: 35.5422930939341

Insight:
Most trips have a relatively low average speed, with a median of about 9.52 mph.
The very high maximum speed shows some extreme values in the data.
```

Outlier Analysis Using IQR

Trip Distance IQR

```
print("IQR Outlier Analysis - Trip Distance")

distance = clean_df['trip_distance'].dropna()

Q1 = np.percentile(distance, 25)
Q3 = np.percentile(distance, 75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

outliers_distance = clean_df[(clean_df['trip_distance'] < lower_bound) |(clean_df['trip_distance'] > uppe

print("\nNumber of potential outliers:", len(outliers_distance))
print('-'*100)
print("\nHighest Trip Distance Records")

print(outliers_distance[['trip_distance', 'trip_duration_minutes', 'PULocationID', 'DOLocationID']].sort_
print('-'*100)
print("Conclusion:")
print("Some extreme trip distance values appear to be data errors.")
print("They are paired with very short trip durations, so they require further investigation.")
```

```
IQR Outlier Analysis - Trip Distance
Q1: 0.98
Q3: 3.1
IQR: 2.12
Lower Bound: -2.2
Upper Bound: 6.28

Number of potential outliers: 422384
----------------------------------------------------------------------------------------------------

Highest Trip Distance Records
         trip_distance  trip_duration_minutes  PULocationID  DOLocationID
3275762      276423.57                    9.0           161           170
3188501      276099.95                    7.0           224           233
3240337      222167.49                   26.0            52           144
```

```
3112173      206137.99                 24.0           263            48
3374854      202771.63                 13.0            50           186
3081597      189687.43                 18.0           246           164
3069039      181139.99                  6.0           100            90
3312563      168079.57                 11.0            42           238
3195037      167452.94                 27.0           247           239
3138510      164959.95                 15.0           262           239
--------------------------------------------------------------------------------------------------
Conclusion:
Some extreme trip distance values appear to be data errors.
They are paired with very short trip durations, so they require further investigation.
```

Trip Duration IQR

```
print("IQR Outlier Analysis - Trip Duration")

duration = clean_df['trip_duration_minutes'].dropna()

Q1 = np.percentile(duration, 25)
Q3 = np.percentile(duration, 75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

outliers_duration = clean_df[
    (clean_df['trip_duration_minutes'] < lower_bound) |
    (clean_df['trip_duration_minutes'] > upper_bound)
]

print("\nNumber of potential outliers:", len(outliers_duration))
print('-'*100)
print("\nHighest Trip Duration Records")

print(outliers_duration[['trip_duration_minutes', 'trip_distance', 'PULocationID', 'DOLocationID']].sort_
print('-'*100)
print("Conclusion:")
print("Some trip durations are extremely high compared to their trip distance.")
print("These records appear to be potential data-quality issues and need further investigation.")
```

```
IQR Outlier Analysis - Trip Duration
Q1: 7.283333333333333
Q3: 18.333333333333332
IQR: 11.049999999999999
Lower Bound: -9.291666666666666
Upper Bound: 34.90833333333333

Number of potential outliers: 191020
--------------------------------------------------------------------------------------------------

Highest Trip Duration Records
         trip_duration_minutes  trip_distance  PULocationID  DOLocationID
1858816            5626.316667           6.55            70            70
1858817            5626.316667           6.55            70           129
414374             5022.133333          91.51           218           193
2460536            4184.000000           0.00           207           226
898201             3405.466667           0.04           132           226
2147536            3062.383333           0.00           207           142
85681              2876.500000           0.00           193           181
1536147            2875.166667           1.98           229           265
556772             2764.533333           1.86           265           141
3417048            2589.333333           7.52           231           193
--------------------------------------------------------------------------------------------------
Conclusion:
Some trip durations are extremely high compared to their trip distance.
These records appear to be potential data-quality issues and need further investigation.
```

Average Speed IQR

```
print("IQR Outlier Analysis - Average Speed")

speed = clean_df['average_speed_mph'].dropna()
```

```
Q1 = np.percentile(speed, 25)
Q3 = np.percentile(speed, 75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)

outliers_speed = clean_df[
    (clean_df['average_speed_mph'] < lower_bound) |
    (clean_df['average_speed_mph'] > upper_bound)
]

print("\nNumber of potential outliers:", len(outliers_speed))
print('-'*100)
print("Highest Average Speed Records")

print(outliers_speed[['average_speed_mph', 'trip_distance', 'trip_duration_minutes','PULocationID', 'DOLc

print('-'*100)
print("Conclusion:")
print("Some average speed values are extremely high and unrealistic.")
print("These records are likely linked to incorrect distance or duration values and need further investig
```

```
IQR Outlier Analysis - Average Speed
Q1: 7.2090112640801
Q3: 12.93291536050157
IQR: 5.723904096421469
Lower Bound: -1.3768448805521025
Upper Bound: 21.51877150513377

Number of potential outliers: 270262
----------------------------------------------------------------------------------------------------
Highest Average Speed Records
         average_speed_mph  trip_distance  trip_duration_minutes  \
3070619       3.722497e+06      124083.23                    2.0
3188501       2.366571e+06      276099.95                    7.0
3107124       2.132599e+06      106629.95                    3.0
3275762       1.842824e+06      276423.57                    9.0
3069039       1.811400e+06      181139.99                    6.0
2993436       1.437123e+06      143712.27                    6.0
3301979       1.055196e+06      105519.62                    6.0
3063201       1.042875e+06       52143.74                    3.0
2945138       9.747746e+05       81231.22                    5.0
3374854       9.358691e+05      202771.63                   13.0

         PULocationID  DOLocationID
3070619            74            75
3188501           224           233
3107124           186            90
3275762           161           170
3069039           100            90
2993436           262            75
3301979            28           130
3063201           113            79
2945138           247           247
3374854            50           186
----------------------------------------------------------------------------------------------------
Conclusion:
Some average speed values are extremely high and unrealistic.
These records are likely linked to incorrect distance or duration values and need further investigation.
```

IQR Outlier Analysis Conclusion:

- The analysis identified extreme values in trip distance, trip duration and average speed.
- Several records appear to be data-quality issues, especially those with unrealistic distance, duration or speed values.
- These records should be investigated rather than automatically deleted.

Phase M — Correlation Analysis

```
corr_matrix = clean_df[
    ['passenger_count',
     'trip_distance',
     'trip_duration_minutes',
     'average_speed_mph']].corr()

print(corr_matrix)
```

```
                     passenger_count  trip_distance  trip_duration_minutes  \
passenger_count             1.000000       0.003391               0.022469
trip_distance               0.003391       1.000000               0.003013
trip_duration_minutes       0.022469       0.003013               1.000000
average_speed_mph           0.000959       0.785068              -0.000339

                     average_speed_mph
passenger_count               0.000959
trip_distance                 0.785068
trip_duration_minutes        -0.000339
average_speed_mph             1.000000
```

```
print("Q.93 Which variables have a strong relationship?")

print("\nTrip distance and average speed have a strong positive relationship.")
print("Correlation:", corr_matrix.loc['trip_distance', 'average_speed_mph'])
```

```
Q.93 Which variables have a strong relationship?

Trip distance and average speed have a strong positive relationship.
Correlation: 0.7850675944732626
```

```
print("Q.94 Which variables have a weak relationship?")

print("\nPassenger count has a very weak relationship with the other variables.")
print("Trip duration also has a very weak relationship with the other variables.")
```

```
Q.94 Which variables have a weak relationship?

Passenger count has a very weak relationship with the other variables.
Trip duration also has a very weak relationship with the other variables.
```

```
print("Q.95 Is trip distance positively related to trip duration?")

print("\nCorrelation:", corr_matrix.loc['trip_distance', 'trip_duration_minutes'])
print("Trip distance and trip duration have a very weak positive relationship.")
```

```
Q.95 Is trip distance positively related to trip duration?

Correlation: 0.0030132613977518455
Trip distance and trip duration have a very weak positive relationship.
```

```
print("Q.96 Is passenger count strongly related to trip distance?")

print("\nCorrelation:", corr_matrix.loc['passenger_count', 'trip_distance'])
print("No, passenger count has almost no relationship with trip distance.")
```

```
Q.96 Is passenger count strongly related to trip distance?

Correlation: 0.0033911171925908377
No, passenger count has almost no relationship with trip distance.
```

```
print("Q.97 Does correlation imply causation in this dataset?")

print("\nNo, correlation does not imply causation.")
print("It only shows the relationship between variables.")
```

```
Q.97 Does correlation imply causation in this dataset?

No, correlation does not imply causation.
It only shows the relationship between variables.
```

```
print("Correlation Heatmap")

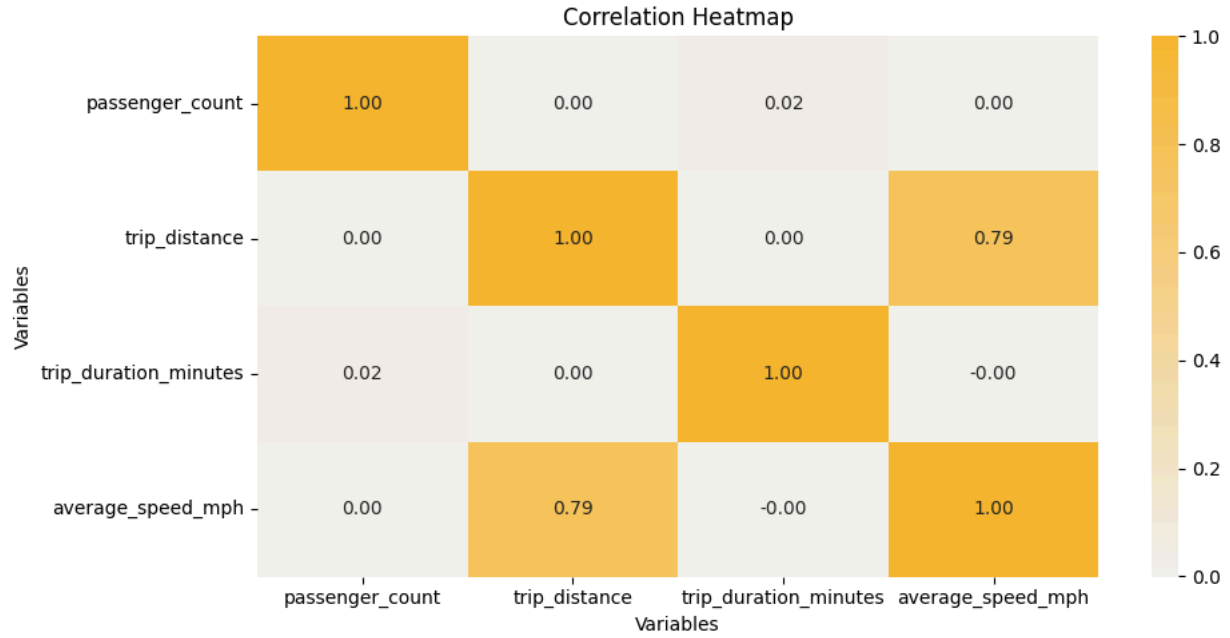
plt.figure(figsize=(10, 5))

sns.heatmap(
```

```
    corr_matrix,
    annot=True,
    cmap=sns.light_palette("#F7B731", as_cmap=True),
    fmt='.2f')

plt.title('Correlation Heatmap')
plt.xlabel('Variables')
plt.ylabel('Variables')
plt.tight_layout()
plt.show()
print('---'*20)
print("Insight-->")
print("Trip distance and average speed show a strong positive relationship (0.79).")
print("Passenger count has very weak relationships with the other variables.")
print("Trip distance and trip duration show almost no correlation in this dataset.")
print("Correlation shows a relationship between variables, but it does not mean causation.")
```

```
Correlation Heatmap
```




```
------------------------------------------------------------
Insight-->
Trip distance and average speed show a strong positive relationship (0.79).
Passenger count has very weak relationships with the other variables.
Trip distance and trip duration show almost no correlation in this dataset.
Correlation shows a relationship between variables, but it does not mean causation.
```

```
print("Hour × Day Trip Activity Heatmap")

hour_day_trips = pd.crosstab(
    clean_df['day_of_week'],
    clean_df['pickup_hour'])

day_order = [
    'Monday', 'Tuesday', 'Wednesday',
    'Thursday', 'Friday', 'Saturday', 'Sunday']

hour_day_trips = hour_day_trips.reindex(day_order)

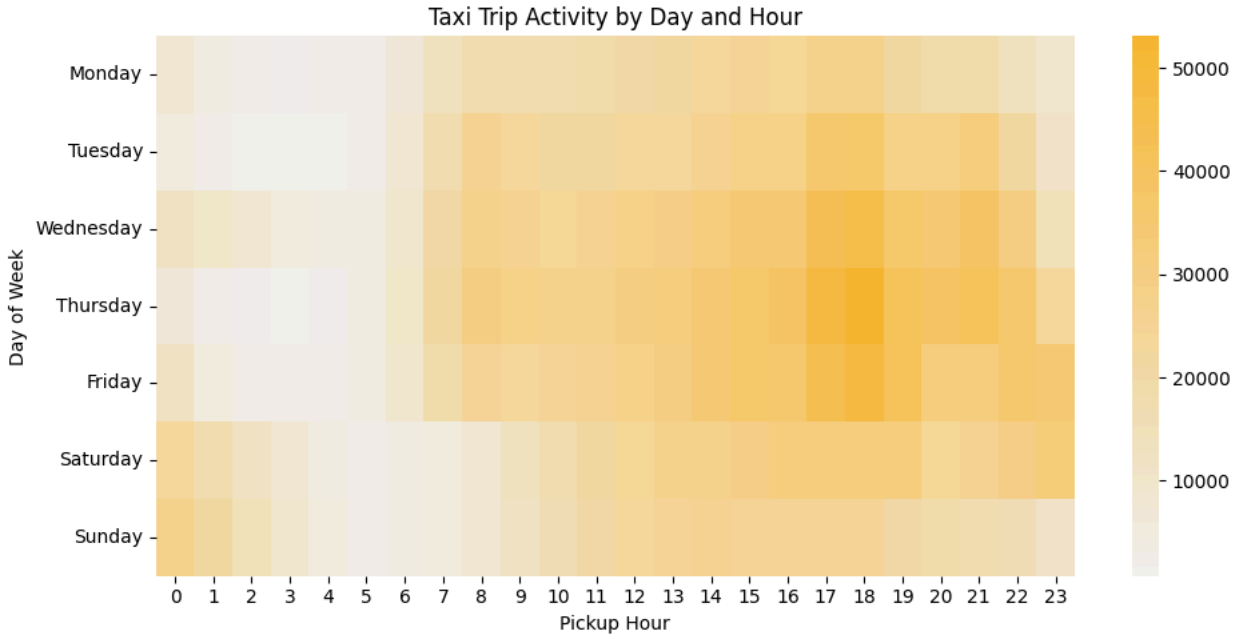
plt.figure(figsize=(10,5))

sns.heatmap(hour_day_trips,cmap=sns.light_palette("#F7B731", as_cmap=True))

plt.title('Taxi Trip Activity by Day and Hour')
plt.xlabel('Pickup Hour')
plt.ylabel('Day of Week')
plt.tight_layout()
plt.show()

print('---'*20)
print("Insight:")
print("Taxi activity is lowest during the early morning hours, especially around 3–5 AM.")
print("Trip activity increases during the day and is highest around the evening hours, especially on Thur
```

```
Hour × Day Trip Activity Heatmap
```




```
------------------------------------------------------------
Insight:
Taxi activity is lowest during the early morning hours, especially around 3–5 AM.
Trip activity increases during the day and is highest around the evening hours, especially on Thursday and Friday.
```

Phase N — Advanced Analysis

```
print("N.2 Top 20 Most Frequently Travelled Routes\n")

top_20_routes = clean_df['route'].value_counts().head(20)

print(top_20_routes)

print('---'*20)
print("A few routes have much higher trip activity than the others.")
```

```
N.2 Top 20 Most Frequently Travelled Routes

route
237 → 236    24839
236 → 237    21843
236 → 236    18221
237 → 237    17441
161 → 237    11463
237 → 161    10168
161 → 236     9845
239 → 238     9595
142 → 239     9141
239 → 142     8844
141 → 236     8816
186 → 230     8549
236 → 141     8246
236 → 161     8037
237 → 162     7885
237 → 141     7867
238 → 239     7854
236 → 239     7689
132 → 132     7596
161 → 161     7492
Name: count, dtype: int64
------------------------------------------------------------
A few routes have much higher trip activity than the others.
```

```
print("N.3 Routes with Highest Average Trip Duration\n")

route_count = clean_df['route'].value_counts()

valid_routes = route_count[route_count >= 100].index

route_duration = clean_df[
```

```
    clean_df['route'].isin(valid_routes)
].groupby('route')['trip_duration_minutes'].mean()

print(route_duration.sort_values(ascending=False).head(10))

print('---'*20)
print("Some routes have higher average trip durations than others.")
print("Only routes with at least 100 trips were considered.")
```

```
N.3 Routes with Highest Average Trip Duration

route
132 → 1      69.189821
246 → 132    56.986967
87 → 132     55.277591
43 → 132     54.877181
158 → 132    54.759159
68 → 132     53.955210
132 → 50     53.841791
132 → 143    53.807826
143 → 132    53.182177
132 → 43     52.509223
Name: trip_duration_minutes, dtype: float64
------------------------------------------------------------
Some routes have higher average trip durations than others.
Only routes with at least 100 trips were considered.
```

```
print("N.4 Average Speed Analysis\n")

print("Lowest Average Speed Records:")
print(clean_df[['average_speed_mph',
                'trip_distance',
                'trip_duration_minutes']].sort_values(
                    'average_speed_mph'
                ).head(10))

print("\nHighest Average Speed Records:")
print(clean_df[['average_speed_mph',
                'trip_distance',
                'trip_duration_minutes']].sort_values(
                    'average_speed_mph',
                    ascending=False
                ).head(10))

print('---'*20)
print("The lowest speed values are mainly caused by trips with zero distance.")
print("The highest speed values are linked to extreme distances or very short durations.")
```

```
N.4 Average Speed Analysis

Lowest Average Speed Records:
         average_speed_mph  trip_distance  trip_duration_minutes
3270651                0.0            0.0              15.833333
3270668                0.0            0.0               8.650000
3014004                0.0            0.0              10.433333
3014024                0.0            0.0               6.800000
3014027                0.0            0.0              21.300000
3014016                0.0            0.0               8.916667
3014017                0.0            0.0              27.716667
3014018                0.0            0.0              17.033333
3014037                0.0            0.0              15.933333
3270648                0.0            0.0               6.800000

Highest Average Speed Records:
         average_speed_mph  trip_distance  trip_duration_minutes
3070619       3.722497e+06      124083.23                    2.0
3188501       2.366571e+06      276099.95                    7.0
3107124       2.132599e+06      106629.95                    3.0
3275762       1.842824e+06      276423.57                    9.0
3069039       1.811400e+06      181139.99                    6.0
2993436       1.437123e+06      143712.27                    6.0
3301979       1.055196e+06      105519.62                    6.0
3063201       1.042875e+06       52143.74                    3.0
2945138       9.747746e+05       81231.22                    5.0
3374854       9.358691e+05      202771.63                   13.0
------------------------------------------------------------
The lowest speed values are mainly caused by trips with zero distance.
The highest speed values are linked to extreme distances or very short durations.
```

```
print("N.5 Distance Band Analysis\n")

print("Trip Count:")
```

```
print(clean_df['distance_category'].value_counts())

print("\nAverage Duration:")
print(clean_df.groupby('distance_category')['trip_duration_minutes'].mean())

print("\nAverage Speed:")
print(clean_df.groupby('distance_category')['average_speed_mph'].mean())

print("\nAverage Passenger Count:")
print(clean_df.groupby('distance_category')['passenger_count'].mean())

print('---'*20)
print("Most trips are in the shorter distance bands.")
print("Trip duration increases across the distance bands.")
print("The long-distance average speed is strongly affected by extreme values.")
```

```
N.5 Distance Band Analysis

Trip Count:
distance_category
Short     2577572
Medium     663225
Long       234305
Name: count, dtype: int64

Average Duration:
distance_category
Short     10.584056
Medium    22.825416
Long      41.926172
Name: trip_duration_minutes, dtype: float64

Average Speed:
distance_category
Short       8.771227
Medium     16.099216
Long      239.524619
Name: average_speed_mph, dtype: float64

Average Passenger Count:
distance_category
Short     1.298498
Medium    1.313439
Long      1.402495
Name: passenger_count, dtype: float64
------------------------------------------------------------
Most trips are in the shorter distance bands.
Trip duration increases across the distance bands.
The long-distance average speed is strongly affected by extreme values.
/tmp/ipykernel_72027/1786632781.py:7: FutureWarning: The default of observed=False is deprecated and will be changed to
  print(clean_df.groupby('distance_category')['trip_duration_minutes'].mean())
/tmp/ipykernel_72027/1786632781.py:10: FutureWarning: The default of observed=False is deprecated and will be changed t
  print(clean_df.groupby('distance_category')['average_speed_mph'].mean())
/tmp/ipykernel_72027/1786632781.py:13: FutureWarning: The default of observed=False is deprecated and will be changed t
  print(clean_df.groupby('distance_category')['passenger_count'].mean())
```

```
print("N.6 Peak vs Off-Peak Analysis\n")

clean_df['peak_period'] = np.where(
    clean_df['pickup_hour'].isin([17, 18, 19, 20]),
    'Peak',
    'Off-Peak')

print("Trip Count:")
print(clean_df['peak_period'].value_counts())

print("\nAverage Distance:")
print(clean_df.groupby('peak_period')['trip_distance'].mean())

print("\nAverage Duration:")
print(clean_df.groupby('peak_period')['trip_duration_minutes'].mean())

print("\nAverage Speed:")
print(clean_df.groupby('peak_period')['average_speed_mph'].mean())

print('---'*20)
print("Peak hours are defined as 5 PM to 8 PM.")
print("This period is compared with all other hours as off-peak.")
```

```
N.6 Peak vs Off-Peak Analysis
```

```
Trip Count:
peak_period
Off-Peak    2537586
Peak         937516
Name: count, dtype: int64

Average Distance:
peak_period
Off-Peak    6.079007
Peak        5.249082
Name: trip_distance, dtype: float64

Average Duration:
peak_period
Off-Peak    15.202189
Peak        14.577035
Name: trip_duration_minutes, dtype: float64

Average Speed:
peak_period
Off-Peak    28.253126
Peak        18.920504
Name: average_speed_mph, dtype: float64
------------------------------------------------------------
Peak hours are defined as 5 PM to 8 PM.
This period is compared with all other hours as off-peak.
```

```
print("N.7 Route Concentration\n")

route_trips = clean_df['route'].value_counts()

top_20_trips = route_trips.head(20).sum()
total_trips = clean_df.shape[0]

top_20_percentage = (top_20_trips / total_trips) * 100

print("Trips in Top 20 Routes:", top_20_trips)
print("Total Trips:", total_trips)
print("Top 20 Route Percentage:", top_20_percentage)

print('---'*20)
print("The top 20 routes account for a relatively small share of all trips.")
print("Most trips are spread across many different routes.")
```

```
N.7 Route Concentration

Trips in Top 20 Routes: 221431
Total Trips: 3475102
Top 20 Route Percentage: 6.371928075780222
------------------------------------------------------------
The top 20 routes account for a relatively small share of all trips.
Most trips are spread across many different routes.
```

Business Quesions - Final Executive Summary

```
print("Q.98 When is taxi demand highest?")

print("Busiest Hour:", hourly_trips.idxmax())
print("Number of Trips:", hourly_trips.max())

print("Busiest Day:", daily_trips.idxmax())
print("Number of Trips:", daily_trips.max())
print('---'*20)
print("Taxi demand is highest around 6 PM.")
print("Thursday has the highest overall trip activity.")
```

```
Q.98 When is taxi demand highest?
Busiest Hour: 18
Number of Trips: 267949
Busiest Day: Thursday
Number of Trips: 614208
------------------------------------------------------------
Taxi demand is highest around 6 PM.
Thursday has the highest overall trip activity.
```

```
print("Q.99 When is taxi demand lowest?")
```

```
print("Least Busy Hour:", hourly_trips.idxmin())
print("Number of Trips:", hourly_trips.min())

print("Least Busy Day:", daily_trips.idxmin())
print("Number of Trips:", daily_trips.min())
print('---'*20)
print("Taxi demand is lowest around 4 AM.")
print("Monday has the lowest overall trip activity.")
```

```
Q.99 When is taxi demand lowest?
Least Busy Hour: 4
Number of Trips: 20031
Least Busy Day: Monday
Number of Trips: 363755
------------------------------------------------------------
Taxi demand is lowest around 4 AM.
Monday has the lowest overall trip activity.
```

```
print("Q.100 How does weekday demand compare with weekend demand?")

print("Weekday Trips:", weekday_trips)
print("Weekend Trips:", weekend_trips)

if weekday_trips > weekend_trips:
    print("Weekday demand is higher than weekend demand.")
else:
    print("Weekend demand is higher than weekday demand.")

print('---'*20)
print("Weekday demand is higher than weekend demand.")
```

```
Q.100 How does weekday demand compare with weekend demand?
Weekday Trips: 2596123
Weekend Trips: 878979
Weekday demand is higher than weekend demand.
------------------------------------------------------------
Weekday demand is higher than weekend demand.
```

```
print("Q.101 What does a typical taxi trip look like in terms of distance and duration?")

print("Median Trip Distance:", clean_df['trip_distance'].median())
print("Median Trip Duration:", clean_df['trip_duration_minutes'].median())

print('---'*20)
print("A typical taxi trip is around 1.67 miles and takes about 11.7 minutes.")
```

```
Q.101 What does a typical taxi trip look like in terms of distance and duration?
Median Trip Distance: 1.67
Median Trip Duration: 11.7
------------------------------------------------------------
A typical taxi trip is around 1.67 miles and takes about 11.7 minutes.
```

```
print("Q.102 What are the most common pickup locations?")

print("Top Pickup Locations:")
print(clean_df['PULocationID'].value_counts().head(10))

print('---'*20)
print("A small number of pickup locations account for a large share of trip demand.")
```

```
Q.102 What are the most common pickup locations?
Top Pickup Locations:
PULocationID
161    169977
237    163702
236    155644
132    146137
230    125828
186    119131
162    117929
142    110584
239     96614
163     95906
Name: count, dtype: int64
------------------------------------------------------------
A small number of pickup locations account for a large share of trip demand.
```

```
print("Q.103 What are the most common drop-off locations?")
```

```
print("Top Drop-off Locations:")
print(clean_df['DOLocationID'].value_counts().head(10))

print('---'*20)
print("A small number of drop-off locations receive a large share of taxi trips.")
```

```
Q.103 What are the most common drop-off locations?
Top Drop-off Locations:
DOLocationID
236    161376
237    149970
161    131258
230    108177
170    100057
142     98981
239     97559
162     93798
141     92675
68      89232
Name: count, dtype: int64
------------------------------------------------------------
A small number of drop-off locations receive a large share of taxi trips.
```

```
print("Q.104 What are the most frequently travelled routes?")

route_trips = (
    clean_df.groupby(['PULocationID', 'DOLocationID'])
    .size()
    .sort_values(ascending=False)
    .head(10)
    .reset_index(name='Number of Trips')
)

print(route_trips)

print('---'*20)
print("A small number of routes account for a large share of recorded trips.")
```

```
Q.104 What are the most frequently travelled routes?
   PULocationID  DOLocationID  Number of Trips
0           237           236            24839
1           236           237            21843
2           236           236            18221
3           237           237            17441
4           161           237            11463
5           237           161            10168
6           161           236             9845
7           239           238             9595
8           142           239             9141
9           239           142             8844
------------------------------------------------------------
A small number of routes account for a large share of recorded trips.
```

```
print("Q.105 Are longer trips generally associated with longer durations?")

print("Correlation:", clean_df['trip_distance'].corr(clean_df['trip_duration_minutes']))

print('---'*20)
print("Trip distance and trip duration show a very weak relationship in this dataset.")
```

```
Q.105 Are longer trips generally associated with longer durations?
Correlation: 0.0030132613977519066
------------------------------------------------------------
Trip distance and trip duration show a very weak relationship in this dataset.
```

```
print("Q.106 Which hours show unusually long or short trip durations?")

print("Longest Average Duration Hour:", avg_duration_by_hour.idxmax())
print("Longest Average Duration:", avg_duration_by_hour.max())

print("Shortest Average Duration Hour:", avg_duration_by_hour.idxmin())
print("Shortest Average Duration:", avg_duration_by_hour.min())

print('---'*20)
print("Trips around 4 PM have the highest average duration.")
print("Trips around the shortest-duration hour have the lowest average duration.")
```

```
Q.106 Which hours show unusually long or short trip durations?
Longest Average Duration Hour: 16
```

```
Longest Average Duration: 16.73921294440528
Shortest Average Duration Hour: 3
Shortest Average Duration: 12.500978086647244
------------------------------------------------------------
Trips around 4 PM have the highest average duration.
Trips around the shortest-duration hour have the lowest average duration.
```

```
print("Q.107 What passenger-count patterns can be observed?")

print("Most Common Passenger Count:")
print(clean_df['passenger_count'].value_counts().head())

print('---'*20)
print("One-passenger trips are the most common.")
print("Higher passenger counts occur much less frequently.")
```

```
Q.107 What passenger-count patterns can be observed?
Most Common Passenger Count:
passenger_count
1.0    2322429
2.0     407761
3.0      91409
4.0      59009
5.0      17786
Name: count, dtype: int64
------------------------------------------------------------
One-passenger trips are the most common.
Higher passenger counts occur much less frequently.
```

```
print("Q.108 Which payment types are most common?")

payment_trips = clean_df['payment_type'].value_counts()

print(payment_trips)

print('---'*20)
print("Credit Card is the most common payment type.")
```

```
Q.108 Which payment types are most common?
payment_type
1    2444387
0     540031
2     390429
4      76481
3      23773
5          1
Name: count, dtype: int64
------------------------------------------------------------
Credit Card is the most common payment type.
```

```
print("Q.109 How do trip characteristics vary across vendors?")

print("Average Trip Distance by Vendor:")
print(clean_df.groupby('VendorID')['trip_distance'].mean())

print("\nAverage Trip Duration by Vendor:")
print(clean_df.groupby('VendorID')['trip_duration_minutes'].mean())

print('---'*20)
print("Trip distance and duration vary across vendors.")
print("Vendors 6 and 7 show unusual values, but they represent very few trips.")
```

```
Q.109 How do trip characteristics vary across vendors?
Average Trip Distance by Vendor:
VendorID
1    3.198076
2    6.592589
6    9.537089
7    1.962902
Name: trip_distance, dtype: float64

Average Trip Duration by Vendor:
VendorID
1    15.698637
2    14.852311
6    41.360737
7     0.000000
Name: trip_duration_minutes, dtype: float64
------------------------------------------------------------
Trip distance and duration vary across vendors.
Vendors 6 and 7 show unusual values, but they represent very few trips.
```

```
print("Q.110 Which rate codes are most frequently used?")

ratecode_trips = clean_df['RatecodeID'].value_counts()

print(ratecode_trips)

print('---'*20)
print("Rate code 1 is used most frequently by a large margin.")
```

```
Q.110 Which rate codes are most frequently used?
RatecodeID
1.0     2756471
2.0       94420
99.0      41958
5.0       26501
3.0        8622
4.0        7092
6.0           7
Name: count, dtype: int64
------------------------------------------------------------
Rate code 1 is used most frequently by a large margin.
```

```
print("Q.111 What unusual or potentially problematic records exist?")

print("Drop-off before pickup:", len(
    clean_df[clean_df['tpep_dropoff_datetime'] < clean_df['tpep_pickup_datetime']]))

print("Zero-duration trips:", len(
    clean_df[clean_df['trip_duration_minutes'] == 0]))

print("Zero/negative distance trips:", len(
    clean_df[clean_df['trip_distance'] <= 0]))

print("Extreme distance records:", len(
    clean_df[clean_df['trip_distance'] > 100]))

print('---'*20)
print("Some records contain unusual distances, durations, or passenger values.")
print("These records may indicate data-quality issues and should be treated carefully.")
```

```
Q.111 What unusual or potentially problematic records exist?
Drop-off before pickup: 0
Zero-duration trips: 1927
Zero/negative distance trips: 90893
Extreme distance records: 162
------------------------------------------------------------
Some records contain unusual distances, durations, or passenger values.
These records may indicate data-quality issues and should be treated carefully.
```

```
print("Q.112 What are the most important data-quality limitations?")

print("Missing values are present in several columns.")
print("Some trips have zero or extreme distance values.")
print("Some trips have unusual or extreme durations and speeds.")
print("Passenger counts also contain unusual values.")

print('---'*20)
print("The main limitations are missing values and unusual distance, duration, and passenger-count records
```

```
Q.112 What are the most important data-quality limitations?
Missing values are present in several columns.
Some trips have zero or extreme distance values.
Some trips have unusual or extreme durations and speeds.
Passenger counts also contain unusual values.
------------------------------------------------------------
The main limitations are missing values and unusual distance, duration, and passenger-count records.
```

```
print("Q.113 What operational questions could a taxi company or transportation analyst investigate next?")

print("Which locations need more taxis during peak hours?")
print("Which routes have consistently high demand?")
print("How does demand change between peak and off-peak hours?")
print("Which trips or routes show unusual travel patterns?")

print('---'*20)
print("These questions can help improve taxi availability, route planning, and peak-hour operations.")
```

```
Q.113 What operational questions could a taxi company or transportation analyst investigate next?
Which locations need more taxis during peak hours?
Which routes have consistently high demand?
How does demand change between peak and off-peak hours?
Which trips or routes show unusual travel patterns?
------------------------------------------------------------
These questions can help improve taxi availability, route planning, and peak-hour operations.
```